CSE 4502

Operating Systems Lab

Lab Handout Book

Building an Operating System Kernel

Seven progressive lab experiments with the mCertiKOS kernel

Experiment 1	Physical Memory Allocator
Experiment 2	Memory Containers and Page Tables
Experiment 3	Virtual Memory Management
Experiment 4	Kernel Threads and Scheduling
Experiment 5	Processes, Traps and System Calls
Experiment 6	Multiprocessor and Synchronization
Experiment 7	Disk and File System

Department of Computer Science and Engineering

Winter 2025–26

Version 1.0

Preface

In this lab you will build the core of a real operating system kernel. You start with a kernel that can boot but cannot manage memory, run programs, or store files. Over seven experiments you add, one layer at a time:

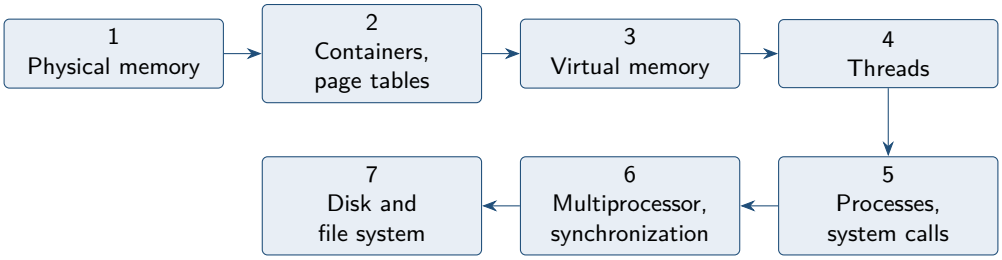
- a physical memory allocator;
- virtual memory;
- threads and a scheduler;
- user processes with system calls;
- multiprocessor synchronization;
- a file system.

At the end, your kernel runs user programs on several CPUs and passes a file system test suite. In the last weeks, your team extends it further in a course project of your choice.

The kernel is *mCertiKOS*, a small teaching version of CertiKOS, an x86 kernel developed at Yale University whose layers were formally verified. You will not write proofs. You will, however, work in the same layered style: every part of the kernel is a small *layer* that may only use the functions of the layers below it. This keeps each task small and self-contained, and it is the main idea to take away from the course.

How the experiments fit together

All seven experiments use **one project**, the `mcertikos` directory. Each experiment builds on the code you wrote in the earlier ones, so you keep working in the same directory for the whole course. The tasks of each experiment are marked in the source code with comments of the form `TODO [Lab n]`. A build option, `LAB`, tells the kernel which experiment you are working on and therefore how far to boot and which tests to run.



How to use this book

- **Before the first lab**, set up your computer by following Chapter I, then read Chapter II, which explains the project, the build system and the testing workflow.
- **Before each lab**, read the whole experiment chapter, in particular the *Background* section, and answer the pre-lab questions. You will not have time to learn the background during the lab.

- **During the lab**, work through the tasks in order. Each task lists the functions to implement, what each function must do, and how to test it. Run the tests often.
- **At the end of the lab**, show the test results and the demonstration to your teacher, and answer the viva questions.

The appendices contain reference material that you will need repeatedly: the x86 facts used by the kernel (Appendix A), the kernel functions available to each layer (Appendix B), debugging techniques (Appendix C), solutions to common problems (Appendix D), and the lab report format (Appendix E).

Conventions

`palloc()`	a function, variable or other identifier in C code
kern/pmm/MATOp/MATOp.c	a file or directory, relative to mcertikos/
`make LAB=1 TEST=1`	a command to type in a terminal
`Ctrl-a` `x`	keys to press: `Ctrl-a`, release, then `x`

Note. Notes give additional explanations.

Tip. Tips suggest a way to save time or to avoid a common mistake.

Caution. Cautions warn about something that will break your kernel or cost you marks.

Contents

Preface i

Lab Rules and Evaluation vii

I Getting Started 1

I Setting Up the Lab Environment 2

I.1 Required tools . . . 2
I.2 Choosing your archive . . . 2
I.3 Ubuntu, Debian and Linux Mint . . . 3
I.4 Fedora . . . 3
I.5 macOS . . . 4
I.6 Windows 10 and 11 (WSL 2) . . . 4
I.7 Virtual machine (any system) . . . 5
I.8 Checking the environment . . . 5
I.9 Setting up version control . . . 6
I.10 Setting up the editor . . . 6

II Working with the mCertiKOS Project 7

II.1 Directory structure . . . 7
II.2 The layered design . . . 7
II.3 How the kernel boots . . . 8
II.4 Selecting the experiment: the `LAB` option . . . 9
II.5 Building and running . . . 9
II.6 How the tests work . . . 10
II.7 Rules for writing kernel code . . . 10
II.8 Workflow for every experiment . . . 10

II Lab Experiments 12

1 Physical Memory Allocator 13

1.1 Objectives . . . 13
1.2 Pre-lab work . . . 13
1.3 Background . . . 13
1.4 Tasks . . . 15

1.5 Testing . . . 16
1.6 Evaluation . . . 17
1.7 Viva questions . . . 17

2 Memory Containers and Page Tables 18
2.1 Objectives . . . 18
2.2 Pre-lab work . . . 18
2.3 Background . . . 18
2.4 Files and interfaces . . . 20
2.5 Tasks . . . 20
2.6 Testing . . . 22
2.7 Evaluation . . . 22
2.8 Viva questions . . . 22

3 Virtual Memory Management 23
3.1 Objectives . . . 23
3.2 Pre-lab work . . . 23
3.3 Background . . . 23
3.4 Files and interfaces . . . 24
3.5 Tasks . . . 25
3.6 Testing . . . 26
3.7 Demonstration: running with paging on . . . 27
3.8 Evaluation . . . 27
3.9 Viva questions . . . 27

4 Kernel Threads and Scheduling 28
4.1 Objectives . . . 28
4.2 Pre-lab work . . . 28
4.3 Background . . . 28
4.4 Files and interfaces . . . 30
4.5 Tasks . . . 30
4.6 Testing . . . 31
4.7 Evaluation . . . 32
4.8 Viva questions . . . 32

5 Processes, Traps and System Calls 33
5.1 Objectives . . . 33
5.2 Pre-lab work . . . 33
5.3 Background . . . 33
5.4 Files and interfaces . . . 35
5.5 Tasks . . . 35

5.6 Testing . . . 37
5.7 Evaluation . . . 38
5.8 Viva questions . . . 38

6 Multiprocessor and Synchronization 39
6.1 Objectives . . . 39
6.2 Pre-lab work . . . 39
6.3 Background . . . 39
6.4 Files and interfaces . . . 41
6.5 Tasks . . . 41
6.6 Testing . . . 43
6.7 Evaluation . . . 44
6.8 Viva questions . . . 44

7 Disk and File System 45
7.1 Objectives . . . 45
7.2 Pre-lab work . . . 45
7.3 Background . . . 45
7.4 Tasks . . . 47
7.5 Testing . . . 49
7.6 Evaluation . . . 50
7.7 Viva questions . . . 50

Appendices

A x86 and mCertiKOS Reference 51
A.1 Important constants . . . 51
A.2 Address translation . . . 51
A.3 Page table entry flags . . . 52
A.4 Registers . . . 52
A.5 Trap numbers . . . 52
A.6 System calls . . . 53
A.7 Error numbers . . . 53
A.8 Thread states . . . 53

B Kernel Function Reference 54
B.1 Physical memory (kern/pmm) . . . 54
B.2 Virtual memory (kern/vmm) . . . 55
B.3 Threads (kern/thread) . . . 56
B.4 Traps and system calls (kern/trap) . . . 56
B.5 File system (kern/fs) . . . 57

B.6 Library (kern/lib) 57

C Debugging the Kernel **58**

C.1 Print debugging 58

C.2 Reading a kernel panic 58

C.3 Debugging with GDB 58

D Troubleshooting **60**

D.1 Build problems 60

D.2 Run-time problems 61

D.3 Getting help 62

E Lab Record and Report Format **63**

E.1 Report structure 63

E.2 Checklist before the evaluation 63

E.3 Experiment record sheet 64

Lab Rules and Evaluation

General rules

1. Come to the lab prepared. Read the experiment chapter and answer the pre-lab questions *before* the lab session.
2. Bring a computer on which the environment of Chapter I already works, or use a lab computer that has it. Setting up the environment is not part of the lab time.
3. Work only in your own copy of the project. Keep the code of all previous experiments: every experiment needs it.
4. Commit your work to version control at the end of every task (Section I.9). A lost or broken tree is not an excuse for a missing experiment.
5. Do not change the provided test files (`test.c`) or the functions outside the `TODO` regions unless the task says so. Your teacher evaluates your work with the original tests.
6. Leave the lab computer as you found it, and log out at the end of the session.

Academic integrity

You may discuss ideas, the background material and the provided code with other students. The code you submit and show must be written by you (or by your group, if your teacher allows groups). Copying code from another student, from a previous year, or from the Internet, and letting others copy your code, are both academic misconduct. In the viva you must be able to explain every line you wrote.

Lab schedule

The lab meets once a week for 100 minutes, for 14 weeks. Each experiment comes after its topics have been taught in the theory course (CSE 4501).

Week	Lab work
1	Orientation; setting up the environment (Chapter I)
2	Warm-up: tour of the project and debugging (Chapter II, Appendix C, optional Task 1.4)
3	Experiment 1: Physical Memory Allocator
4	Experiment 2: Memory Containers and Page Tables
5	Experiment 3: Virtual Memory Management
6	Experiment 4: Kernel Threads and Scheduling
7–8	Experiment 5: Processes, Traps and System Calls (two sessions)
9–10	Experiment 6: Multiprocessor and Synchronization (two sessions)
11–12	Experiment 7: Disk and File System (two sessions); project teams formed in week 12
13	Course project: proposal and progress review
14	Course project: demonstration and presentation; final lab test

How each experiment is evaluated

Each experiment is evaluated in the lab, at the end of its last session (Experiments 5, 6 and 7 take two sessions), in three parts.

Automatic tests
Running `make LAB=n TEST=1` boots your kernel and runs the tests of experiment n. The last lines show the result:

```
================================================
LAB 3 RESULT: 4 / 4 test groups passed.
================================================
```

Each test group is worth the marks given in the experiment's evaluation table.

Demonstration
Experiments 3, 5, 6 and 7 also require a demonstration, such as running user processes. The expected output is given in the experiment.

Viva
Your teacher asks you some of the viva questions at the end of the experiment, or similar questions about your code.

Every experiment is marked out of 10. Your teacher may adjust the weights and the deadlines; the evaluation table of each experiment is the default.

Exp.	Title	Evaluated by	Marks
1	Physical Memory Allocator	3 test groups, viva	10
2	Memory Containers and Page Tables	2 test groups, viva	10
3	Virtual Memory Management	4 test groups, demonstration, viva	10
4	Kernel Threads and Scheduling	4 test groups, viva	10
5	Processes, Traps and System Calls	2 test groups, demonstration, viva	10
6	Multiprocessor and Synchronization	3 test groups, demonstration, viva	10
7	Disk and File System	file system test suite, viva	10

Course assessment

Component	Details	Weight
Attendance	class participation	10%
Lab experiments	Experiments 1–7, each marked out of 10 as above	50%
Course project	proposal, progress review, demonstration and report	30%
Final lab test / viva	individual, in week 14	10%

Course project

After the experiments, teams of students extend the kernel with a new feature, chosen from a list of projects (for example a UNIX shell, a scheduler, or file locking). The project is built on top of your completed Experiment 7 with `make LAB=8`. The projects, their requirements, the deliverables and the marking rubric are described in a separate document, the *CSE 4502 Project Description*, given out in week 12.

Passing tests is necessary, not sufficient. The tests check the most important behaviour, not everything. Code that passes the tests but breaks a later experiment must still be fixed. Code that passes the tests by special-casing them earns no marks.

PART I

Getting Started

CHAPTER I

Setting Up the Lab Environment

The mCertiKOS kernel is a 32-bit x86 program. You compile it with GCC on your own computer and run it inside *QEMU*, an emulator of a complete PC. This chapter explains how to install the required tools, how to get the project, and how to check that everything works. Complete it before your first lab session.

I.1 Required tools

Tool	Purpose
GCC with 32-bit support	compiles the kernel and the user programs for 32-bit x86 (`-m32`)
GNU binutils (`ld`, `objdump`, `nm`)	links the kernel and produces disassembly listings
GNU Make 4	runs the build (Makefile)
Python 3 and Perl	build the bootable disk image and check the boot loader size
QEMU (`qemu-system-x86_64`)	emulates the PC that runs your kernel
GDB	debugs the kernel while it runs in QEMU
Git	keeps the history of your work (strongly recommended)
A text editor	e.g. Visual Studio Code, Vim or Emacs

I.2 Choosing your archive

The project is published on the course homepage (Google Classroom) as one archive per host operating system. The kernel code is the same in all of them; each archive adds a guide (SETUP.md) and an installation script (setup.sh) for its system. Download the one for your computer:

Your computer	Archive	Follow
Ubuntu, Debian, Linux Mint	CSE4502_OS_Lab_Winter2025-26_Ubuntu-Debian.tar.gz	Section I.3
Fedora (RHEL, Rocky, Alma)	CSE4502_OS_Lab_Winter2025-26_Fedora.tar.gz	Section I.4
macOS (Intel or Apple Silicon)	CSE4502_OS_Lab_Winter2025-26_macOS.tar.gz	Section I.5
Windows 10 or 11	CSE4502_OS_Lab_Winter2025-26_Windows-WSL.zip	Section I.6

The recommended platform is **Ubuntu Linux 22.04 or 24.04**; the lab computers run Ubuntu. If nothing else works, use a virtual machine (Section I.7). Every archive contains:

- CSE4502_OS_Lab/SETUP.md: the setup guide for your system;
- CSE4502_OS_Lab/setup.sh: installs the tools and checks them;
- CSE4502_OS_Lab/mcertikos/: the project you work on for the whole course;

- CSE4502_OS_Lab/CSE4502_Lab_Handout_Book.pdf: this book.

> **Caution.** Use a path without spaces for the project (for example ~/os-lab/CSE4502_OS_Lab/mcertikos). Some tools handle spaces in paths badly.

I.3 Ubuntu, Debian and Linux Mint

1. Unpack the archive in your home directory:

```
mkdir -p ~/os-lab
cd ~/os-lab
tar xzf ~/Downloads/CSE4502_OS_Lab_Winter2025-26_Ubuntu-Debian.tar.gz
cd CSE4502_OS_Lab
```

2. Install the tools with the script, which also checks the result:

```
bash setup.sh
```

   Or install them by hand:

```
sudo apt update
sudo apt install build-essential gcc-multilib gdb make perl python3 \
                 qemu-system-x86 git
```

   `gcc-multilib` provides the 32-bit C runtime support that the kernel needs. Without it, linking fails with errors such as `undefined reference to '__udivdi3'`.

3. Check the 32-bit compiler support (the script does this too). The printed path must contain /32/:

```
gcc -m32 -print-libgcc-file-name
```

```
/usr/lib/gcc/x86_64-linux-gnu/13/32/libgcc.a
```

4. *(Recommended)* Enable hardware acceleration (KVM). With KVM, QEMU runs your kernel at nearly native speed; without it, QEMU emulates every instruction and is much slower, though still usable. The script adds you to the `kvm` group when /dev/kvm exists; log out and log in again afterwards. If /dev/kvm does not exist, enable *virtualization* (Intel VT-x or AMD-V) in your computer's BIOS/UEFI settings. The build uses KVM automatically whenever /dev/kvm exists.

I.4 Fedora

1. Unpack the archive:

```
mkdir -p ~/os-lab
cd ~/os-lab
tar xzf ~/Downloads/CSE4502_OS_Lab_Winter2025-26_Fedora.tar.gz
cd CSE4502_OS_Lab
```

2. Install the tools with `bash setup.sh`, or by hand:

```
sudo dnf install gcc make perl python3 qemu-system-x86 gdb git \
                 glibc-devel.i686 libgcc.i686
```

The two `.i686` packages provide the 32-bit support for `gcc -m32`.

3. Enable KVM as described for Ubuntu (the script adds you to the `kvm` group).

> **Note.** Fedora ships very recent compilers (GCC 14 and 15) and linkers. The project is set up for them: it compiles as C11 and builds the boot loader correctly with the newer linker. If you have an *old* copy of the project that fails with `boot block too large` or `'bool' cannot be defined via typedef`, download the current archive (Appendix D).

I.5 macOS

macOS cannot produce 32-bit x86 kernels with its own compiler, so you install a *cross-compiler* for the `i686-elf` target; the build finds it automatically. This works on Intel and Apple Silicon Macs.

1. Install the Homebrew package manager from `https://brew.sh`.
2. Unpack the archive in the Terminal app:

```
mkdir -p ~/os-lab
cd ~/os-lab
tar xzf ~/Downloads/CSE4502_OS_Lab_Winter2025-26_macOS.tar.gz
cd CSE4502_OS_Lab
```

3. Install the tools with `bash setup.sh`, or by hand:

```
brew install i686-elf-binutils i686-elf-gcc qemu make python git
brew install i386-elf-gdb      # optional, for debugging
echo 'export PATH="$(brew --prefix)/opt/make/libexec/gnubin:$PATH"' \
     >> ~/.zshrc
```

The last line makes `make` run GNU Make 4 from Homebrew: the `make` that ships with macOS (version 3.81) is too old. Open a new terminal afterwards.

4. Check the installation:

```
i686-elf-gcc --version
make --version          # must say GNU Make 4.x
qemu-system-x86_64 --version
```

> **Note.** macOS has no KVM, so QEMU emulates the processor. Everything works, but booting and running the tests take longer than on Linux, especially in experiments 6 and 7. Wait at least a minute before deciding that the tests hang.

I.6 Windows 10 and 11 (WSL 2)

The Windows Subsystem for Linux (WSL 2) runs a real Ubuntu system inside Windows. You unpack, build and run the project inside Ubuntu, not in Windows.

1. Open *PowerShell* as administrator and run:

   ```
   wsl --install -d Ubuntu-24.04
   ```

   Restart the computer when asked. On the first start, Ubuntu asks for a user name and a password.

2. Open the *Ubuntu* application and unpack the archive from your Windows Downloads folder into the Linux home directory (replace `YourWindowsName`):

   ```
   sudo apt update && sudo apt install unzip
   mkdir -p ~/os-lab
   cd ~/os-lab
   unzip /mnt/c/Users/YourWindowsName/Downloads/CSE4502_OS_Lab_Winter2025-26
       _Windows-WSL.zip
   cd CSE4502_OS_Lab
   ```

   Keep the project inside the Linux file system, not under /mnt/c/: building on the Windows drive is much slower and loses file permissions.

3. Install the tools with `bash setup.sh` (or as in Section I.3).

4. Run the kernel with `make qemu-nox`, which uses the terminal instead of a graphical window. With Visual Studio Code and its *WSL* extension, `code .` in the Ubuntu terminal opens the project.

> **Note.** On Windows 11, WSL 2 usually provides /dev/kvm. If it does not, QEMU still works without acceleration, only more slowly. To enable it, turn on virtualization in the BIOS/UEFI, add `nestedVirtualization=true` under `[wsl2]` in C:\textbackslash Users\textbackslash YourWindowsName\textbackslash .wslconfig, and run `wsl -shutdown`.

I.7 Virtual machine (any system)

If none of the above works, install VirtualBox (`https://www.virtualbox.org`) or VMware Workstation Player, create a virtual machine with at least 2 CPUs, 4 GB of memory and 25 GB of disk, install Ubuntu 24.04 in it, and follow Section I.3 inside the virtual machine with the Ubuntu archive.

I.8 Checking the environment

After `bash setup.sh` reports `Environment ready`, run the tests of experiment 1 in the fresh project:

```
cd ~/os-lab/CSE4502_OS_Lab/mcertikos
ls
make LAB=1 TEST=1
```

```
Makefile  README  boot  kern  labs  make_image.py  misc  newfs  scripts  user
```

The command compiles the boot loader, the kernel and the user programs, builds the disk image certikos.img, and starts QEMU. The kernel boots and runs the tests of experiment 1.

Because you have not written any code yet, the tests fail. That is expected; what matters is that you see the result line:

```
Testing the MATIntro layer...
test 1 passed.
test 2.3 failed: (0 != 1 || 0 != 0)
...
================================================
LAB 1 RESULT: 0 / 3 test groups passed.
================================================

Test complete. Please Use Ctrl-a x to exit qemu.
```

Press `Ctrl-a`, release both keys, then press `x` to exit QEMU.

> **Tip.** If you see the `RESULT` line, your environment is ready. If not, look up the error message in Appendix D.

I.9 Setting up version control

Because every experiment depends on the previous ones, losing your code is costly. Put the project under Git once, and commit at the end of every task:

```
cd ~/os-lab/CSE4502_OS_Lab/mcertikos
git init
printf 'obj/\ncertikos.img\ncertikos_disk.img\n.gdbinit\n' > .gitignore
git add .
git commit -m "Initial project"

# after finishing a task:
git add -A
git commit -m "Lab 1: implement palloc"
```

`git log` shows your history, `git diff` shows the changes since the last commit, and `git checkout - <file>` restores the last committed version of a file.

> **Caution.** Do not publish your solutions in a public repository. A private repository on GitHub or GitLab is a good backup.

I.10 Setting up the editor

Any editor works. With Visual Studio Code, install the *C/C++* extension and open the `mcertikos` directory with *File → Open Folder*. To jump to the definition of a function, use `F12`. On the command line, `grep -rn name kern` finds every use of `name`. `make cscope` builds an index for the `cscope` code browser.

CHAPTER II

Working with the mCertiKOS Project

This chapter is a guided tour of the project. It explains:

- how the source code is organised;
- how the kernel boots;
- how to build and run it for each experiment;
- how the tests work;
- the rules for writing kernel code.

Read it before the first experiment and come back to it whenever needed.

II.1 Directory structure

Directory	Contents
boot/	the two-stage boot loader (boot0, boot1); provided
kern/init/	kernel entry point and `kern_init`, which selects what to run for each experiment
kern/lib/	kernel library: `dprintf`, strings, spinlocks, x86 helper functions, monitor
kern/dev/	device drivers: console, serial port, timer, interrupt controllers, CPUs, disk; provided
kern/pmm/	physical memory management (experiments 1 and 2)
kern/vmm/	virtual memory management (experiments 2 and 3)
kern/thread/	threads and scheduling (experiments 4 and 6)
kern/proc/	user processes (provided)
kern/trap/	traps, interrupts and system calls (experiments 5 and 6)
kern/fs/	the file system (experiment 7)
user/	user programs (idle, pingpong, fstest, fork_test) and the user library
labs/	short summaries of the seven experiments
newfs/	the initial file system disk image used in experiment 7
obj/	build output (created by `make`; never edit it)

II.2 The layered design

The kernel is a stack of small *layers*. Each layer lives in its own directory, for example kern/pmm/MATIntro/, and contains:

<Layer>.c	the implementation; this is where your `TODO`s are
import.h	the functions of *lower* layers that this layer may call
export.h	the functions this layer offers to *higher* layers
test.c	the tests of the layer (compiled only with `TEST=1`)
Makefile.inc	build rules

> **Use only the functions in import.h.** A layer must not access the data of another layer directly. For example, MATOp.c must not touch the array `AT` defined in MATIntro.c; it uses the getters and setters `at_is_norm`, `at_set_allocated` and so on. Respecting this rule keeps every task small and makes bugs easy to locate.

Figure II.1 shows the layers you implement, grouped by experiment. Each layer is built on the layers below it.

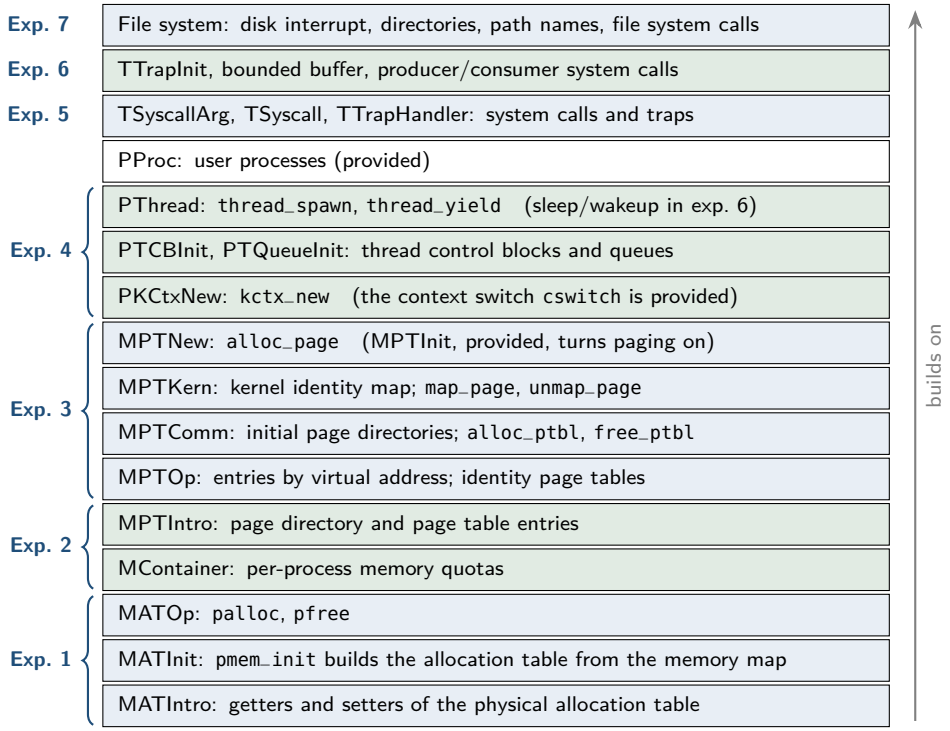


Figure II.1: The layers of mCertiKOS that you implement, from the bottom up. Each layer uses only the layers below it.

II.3 How the kernel boots

1. The BIOS loads the first stage of the boot loader (boot0) from the first sector of the disk.
2. boot0 loads boot1, which switches the CPU to 32-bit protected mode, reads the memory map from the BIOS, loads the kernel at physical address 1 MB, and jumps to it.
3. The kernel entry code calls `kern_init` in kern/init/init.c. Depending on `LAB`, `kern_init` initializes the kernel up to the layer of the current experiment and then runs the tests (`TEST=1`), the kernel monitor, or user programs.

Each layer's initialization function first calls the initialization function of the layer below it. For example:

- `thread_init` calls `tqueue_init`;
- `tqueue_init` calls `tcb_init`;
- `tcb_init` calls `paging_init`;
- and so on, down to `devinit`, which initializes the devices.

You will see this pattern in the code you complete.

II.4 Selecting the experiment: the LAB option

The build option `LAB` tells the kernel which experiment you are working on (1 to 7), or that you are working on the course project (8):

LAB	The kernel initializes up to	Then runs
1	`pmem_init`	tests of exp. 1, or the monitor
2	`container_init`	tests of exp. 2, or the monitor
3	`paging_init` (paging on)	tests of exp. 3, or the monitor
4	`thread_init`	tests of exp. 4, or the monitor
5	`thread_init`	tests of exp. 5, or the monitor (type `startuser`)
6	all CPUs	tests of exp. 6, or the producer/consumer programs
7	all layers	the file system test program `fstest`
8	all layers (as for 7)	your course project: the code in the `LAB == 8` block of user/idle/idle.c

Give `LAB` on the command line, for example `make LAB=3`, or change the default line `LAB ?= 1` in the Makefile when you start a new experiment. When `LAB` or `TEST` changes, the build automatically starts again from a clean state.

> **Note.** Experiment n needs all experiments before it. For example, the tests of experiment 4 cannot run if your physical memory allocator from experiment 1 does not work.

II.5 Building and running

Command	Effect
`make LAB=n`	compiles everything and builds the disk image certikos.img
`make LAB=n TEST=1`	compiles with the tests of experiment n and runs them in QEMU
`make qemu-nox`	runs the last built kernel in QEMU, in the terminal
`make qemu`	the same, with a QEMU window (the output appears in both)
`make qemu-nox-gdb`	runs QEMU and waits for GDB (Appendix C)
`make clean`	deletes all build output

To exit QEMU, press `Ctrl-a` then `x`.

QEMU emulates a PC with 4 CPUs, 2 GB of memory and two disks:

- certikos.img, which holds the boot loader and the kernel;
- certikos_disk.img, which holds the file system.

The kernel prints its output on the emulated serial port, which QEMU connects to your terminal.

The kernel monitor

When no tests or programs are selected, the kernel starts a small interactive *monitor*:

```
***************************************
Welcome to the mCertiKOS kernel monitor!
***************************************
Type 'help' for a list of commands.
$> help
help - Display this list of commands
kerninfo - Display information about the kernel
backtrace - Display a backtrace of the kernel stack
startuser - Start the user idle process (lab 5)
```

II.6 How the tests work

Each layer has a test.c with functions such as `MATOp_test1`. With `TEST=1`, `kern_init` runs the test groups of the current experiment. Each group prints `test k passed` or a message saying what went wrong, for example:

```
test 1.1 failed: (0 < VM_USERLO_PI || VM_USERHI_PI <= 0)
```

The failure message usually contains the condition that was checked. Open the test.c file and find the test (here, test 1.1 of kern/pmm/MATOp/test.c) to see exactly what it expects. Every test.c also has an empty function `..._test_own` where you may add your own tests.

II.7 Rules for writing kernel code

The kernel is written in C without the standard C library, so `printf`, `malloc` and `<string.h>` are not available. Instead, use the kernel's own functions:

Function	Use
`dprintf(fmt, ...)`	print a message, like `printf` (`%d %u %x %s %c %p`)
`KERN_DEBUG(fmt, ...)`	print a message with the file and line number
`KERN_PANIC(fmt, ...)`	print a message and stop the kernel
`KERN_ASSERT(cond)`	stop the kernel if `cond` is false
`memset`, `memzero`, `memcpy`, `memmove`, `memcmp`	memory operations (lib/string.h)
`strcmp`, `strncmp`, `strncpy`, `strnlen`	string operations (lib/string.h)
`ROUNDDOWN(a, n)`, `ROUNDUP(a, n)`	round `a` to a multiple of `n` (lib/types.h)

- Write code only in the `TODO` regions and keep the provided code around them (for example the `spinlock_acquire` and `spinlock_release` calls).
- Do not use large local arrays: a kernel stack is only a few kilobytes.
- Keep the coding style of the surrounding code: four-space indentation and the same naming style.
- All tasks are in C. You never need to write assembly code.

II.8 Workflow for every experiment

1. Read the experiment chapter, answer the pre-lab questions.

2. List the tasks: `grep -rn "TODO \[Lab n\]" kern user`.
3. For each task: read the comments above the function and the layer's import.h, write the code, build, and run `make LAB=n TEST=1`.
4. Commit after each working task.
5. Run the demonstration, if the experiment has one.
6. Show the result to your teacher and answer the viva questions.

PART II

Lab Experiments

EXPERIMENT 1

Physical Memory Allocator

Experiment at a glance

Goal: Build the table that describes every physical page of the machine, and allocate and free physical pages.
Prerequisites: Chapters I and II; C pointers, arrays and bit operations.
Files: kern/pmm/MATIntro/MATIntro.c, kern/pmm/MATInit/MATInit.c, kern/pmm/MATOp/-MATOp.c
Run the tests: `make LAB=1 TEST=1`
Schedule: one lab session of 100 minutes (lab week 3); evaluated at the end of the session
Marks: 10 (tests 8, viva 2)

1.1 Objectives

After this experiment you will be able to:

- explain how the kernel learns which physical memory exists;
- describe the physical memory of the machine as an array of pages;
- implement a page allocator and improve it with memoization.

1.2 Pre-lab work

Read Sections 1.3.1 to 1.3.5, then answer:

1. How many 4 KB pages are there in 4 GB of memory? How many bits are needed to number them?
2. What is the page index of the physical address `0x40000000`? Of `0x00123456`?
3. A memory range starts at `0x9F000` and is `0x1000` bytes long. Which page(s) does it cover completely?

1.3 Background

1.3.1 Physical pages

The kernel manages physical memory in units of *pages* of $4096 = 2^{12}$ bytes (`PAGESIZE`). Page number i (its *page index*) covers the physical addresses $[\, i \cdot 4096,\ (i+1) \cdot 4096 \,)$, so:

$$\text{page index} = \left\lfloor \frac{\text{address}}{4096} \right\rfloor = \text{address} \gg 12, \qquad \text{address of page } i = i \ll 12.$$

A 32-bit machine can address 4 GB, that is at most 2^{20} pages.

1.3.2 The physical memory map

Not all physical addresses are usable memory. Some ranges belong to the BIOS or to devices, and the machine may have less than 4 GB. At boot, the boot loader asks the BIOS for the *memory map*: a table of address ranges, each marked usable or reserved. In QEMU with 2 GB of memory, it looks like this (the kernel prints it at boot):

```
BIOS-e820: 0x00000000 - 0x0009fbff (usable)
BIOS-e820: 0x0009fc00 - 0x0009ffff (reserved)
BIOS-e820: 0x000f0000 - 0x000fffff (reserved)
BIOS-e820: 0x00100000 - 0x7ffdffff (usable)
BIOS-e820: 0x7ffe0000 - 0x7fffffff (reserved)
BIOS-e820: 0xfeffc000 - 0xfeffffff (reserved)
BIOS-e820: 0xfffc0000 - 0xffffffff (reserved)
```

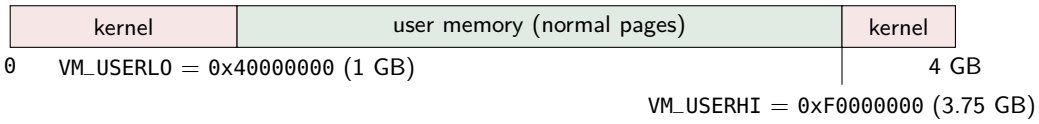
The table is already stored by the device layer. You read it with four functions:

`get_size()`	number of rows in the table
`get_mms(i)`	start address of row `i` (*memory map start*)
`get_mml(i)`	length in bytes of row `i` (*memory map length*)
`is_usable(i)`	1 if row `i` is usable, 0 if it is reserved

The rows are not sorted and are not aligned to pages.

1.3.3 Address space layout

mCertiKOS divides the 4 GB address space into three parts:



Pages below `VM_USERLO` and at or above `VM_USERHI` are reserved for the kernel. In page indices:

- `VM_USERLO_PI` = `0x40000` = 262144;
- `VM_USERHI_PI` = `0xF0000` = 983040.

1.3.4 The allocation table (AT)

The kernel keeps one entry per physical page in the array `AT` (MATIntro.c):

```
struct ATStruct {
    unsigned int perm;       // 0: reserved by the BIOS, 1: kernel only, >1: normal
    unsigned int allocated;  // 0: free, >0: allocated
};
static struct ATStruct AT[1 << 20];
static unsigned int NUM_PAGES;   // number of physical pages (nps)
```

A page is *normal* if its permission is greater than 1. Only normal pages are given to processes.

1.3.5 The layers of this experiment

Layer	Role
MATIntro	owns `AT` and `NUM_PAGES`; offers getters and setters
MATInit	`pmem_init`: computes the number of pages and fills `AT` from the memory map
MATOp	`palloc` and `pfree`: allocate and free pages using the getters and setters

1.4 Tasks

Task 1.1: The allocation table interface (MATIntro)

File: kern/pmm/MATIntro/MATIntro.c. Implement the four functions below. Each one is one or a few lines.

```
unsigned int at_is_norm(unsigned int page_index);
```
Return 1 if the permission of the page is greater than 1 (normal), and 0 otherwise.

```
void at_set_perm(unsigned int page_index, unsigned int perm);
```
Set the permission of the page to `perm` *and* mark the page as unallocated.

```
unsigned int at_is_allocated(unsigned int page_index);
```
Return 1 if the page is allocated (the flag is greater than 0), and 0 otherwise.

```
void at_set_allocated(unsigned int page_index, unsigned int allocated);
```
Set the allocation flag of the page to `allocated`.

Check: `make LAB=1 TEST=1`; the *MATIntro* group must pass.

Task 1.2: Initializing the allocation table (MATInit)

File: kern/pmm/MATInit/MATInit.c, function `pmem_init`. The provided code calls `devinit` and `mem_spinlock_init` first, and `set_nps(nps)` in the middle. Fill in the two `TODO` parts.

1. **Number of pages.** `nps` is the highest end address of any row of the memory map, divided by `PAGESIZE`. Loop over all rows; the end address of row `i` is `get_mms(i) + get_mml(i)`. Keep the maximum, then divide by `PAGESIZE` (rounding down).
2. **Permissions.** For every page index `i` from 0 to `nps - 1`:
   - if `i < VM_USERLO_PI` or `i >= VM_USERHI_PI`, set its permission to 1 (kernel);
   - otherwise, set its permission to 2 if there is a *usable* row of the memory map that contains the *whole* page, that is `start <= i * PAGESIZE` and `(i + 1) * PAGESIZE <= start + length`; otherwise set it to 0.

   Use `at_set_perm`, which also marks the page unallocated.

> **Tip.** Declare all your variables as `unsigned int`. In the QEMU memory map, the last range ends exactly at 4 GB. Its 32-bit end address therefore wraps around to 0 and does not count as the maximum. This is fine: the result is still correct for all usable memory.

Check: the *MATInit* group must pass. It checks that `nps` is larger than 1000, that all kernel pages have permission 1, and that all other pages have permission 0 or 2.

Task 1.3: Allocating and freeing pages (MATOp)

File: kern/pmm/MATOp/MATOp.c.

```
unsigned int palloc(void);
```

Find a page that is normal (`at_is_norm`) and not allocated (`at_is_allocated`), mark it allocated with `at_set_allocated(i, 1)`, and return its index. Return 0 if no such page exists. Only pages in [`VM_USERLO_PI`, `VM_USERHI_PI`) can be normal, so there is no need to search outside this range.

```
void pfree(unsigned int pfree_index);
```

Mark the page as unallocated.

Implement `palloc` in two steps:

1. **First version:** a linear scan from `VM_USERLO_PI` to the first free normal page.
2. **Memoization:** keep a `static` variable with the index where the last search stopped and start the next search there, wrapping around to `VM_USERLO_PI` at the end of the range. Stop when the search returns to its starting point without finding a page.

> **Caution.** The skeleton calls `mem_lock()` at the start and `mem_unlock()` at the end, and returns the variable `palloc_free_index`. Write your code between the two calls and store the result in that variable. Never `return` between `mem_lock()` and `mem_unlock()`: the lock would stay taken and the next call would wait forever. (The lock matters when several CPUs allocate at the same time, in experiment 6.)

Check: the *MATOp* group must pass.

Task 1.4: Optional extensions

- In kern/lib/printfmt.c, implement the `%o` format (unsigned octal). Look at how `%u` is implemented just above it.
- In kern/lib/monitor.c, implement the `backtrace` monitor command. `read_ebp()` returns the current frame pointer. At each frame, `ebp[0]` is the caller's `ebp` and `ebp[1]` is the return address. Print the chain until `ebp` is 0.

1.5 Testing

```
make LAB=1 TEST=1
```

```
Testing the MATIntro layer...
test 1 passed.
test 2 passed.
test 3 passed.
All tests passed.

Testing the MATInit layer...
test 1 passed.
All tests passed.

Testing the MATOp layer...
test 1 passed.
All tests passed.
```

```
================================================
LAB 1 RESULT: 3 / 3 test groups passed.
================================================
```

You can also run `make LAB=1` and `make qemu-nox` and try the monitor commands `help` and `kerninfo`.

1.6 Evaluation

Item	Requirement	Marks
MATIntro	test group passes	2
MATInit	test group passes	3
MATOp	test group passes and `palloc` uses memoization	3
Viva	correct answers to the viva questions	2
	Total	**10**

1.7 Viva questions

1. Why does `palloc` not need to look at pages below `VM_USERLO_PI`?
2. A usable range covers only half of a page. Which permission does the page get, and why?
3. What is the worst-case running time of `palloc` with and without memoization?
4. Why does `at_set_perm` also clear the allocation flag?
5. What happens if two CPUs execute `palloc` at the same time without the lock?

EXPERIMENT 2

Memory Containers and Page Tables

Experiment at a glance

Goal: Limit the memory each process may use (containers), and implement the basic operations on x86 page directories and page tables.
Prerequisites: Experiment 1 complete.
Files: kern/pmm/MContainer/MContainer.c, kern/vmm/MPTIntro/MPTIntro.c
Run the tests: `make LAB=2 TEST=1`
Schedule: one lab session of 100 minutes (lab week 4); evaluated at the end of the session
Marks: 10 (tests 8, viva 2)

2.1 Objectives

After this experiment you will be able to:

- enforce a memory quota per process and split it between a parent and its children;
- explain the two-level page table structure of 32-bit x86;
- build page directory entries and page table entries with the right permission bits.

2.2 Pre-lab work

Read the background, then answer:

1. Split the virtual address `0x4B0C3123` into its page directory index, page table index and offset.
2. Which page directory entry value maps physical page 100 with the bits `PTE_P`, `PTE_W` and `PTE_U` set? Give it in decimal and in hexadecimal.
3. Process 0 has 3 children. What are their container indices? (`MAX_CHILDREN` is 3.)

2.3 Background

2.3.1 Containers

To stop one process from using all the memory, every process has a *container* with a memory *quota* (the maximum number of pages it may use) and a *usage* (the number of pages it uses now):

```
struct SContainer {
    int quota;      // maximum memory quota of the process
    int usage;      // the current memory usage of the process
    int parent;     // the id of the parent process
    int nchildren;  // the number of child processes
```

```
    int used;       // whether current container is used by a process
};
static struct SContainer CONTAINER[NUM_IDS];   // NUM_IDS = 64
```

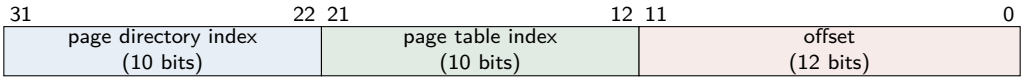
Process 0 (the kernel) starts with a quota equal to all free normal pages. When a process creates a child with quota q, the parent's usage grows by q (the pages are reserved for the child) and the child starts with usage 0 and quota q. Every process may have at most `MAX_CHILDREN` $= 3$ children. The child of process `id` with number `nc` (counting from 0) gets the container index

$$\texttt{child} = \texttt{id} \times \texttt{MAX_CHILDREN} + 1 + \texttt{nc},$$

so the processes form a tree in which each id is unique: the children of 0 are 1, 2, 3; the children of 1 are 4, 5, 6; and so on.

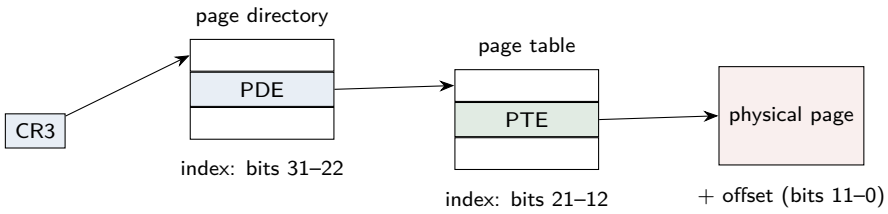
2.3.2 Two-level page tables

With paging, every memory access uses a *virtual* address, which the processor translates to a physical address through the *page structure* of the running process. On 32-bit x86, a virtual address has three parts:



$$\text{PDE index} = va \gg 22, \qquad \text{PTE index} = (va \gg 12)\ \&\ \texttt{0x3FF}, \qquad \text{offset} = va\ \&\ \texttt{0xFFF}.$$

The register `CR3` holds the physical address of the *page directory*: an array of 1024 page directory entries (PDEs). Each PDE points to a *page table*: an array of 1024 page table entries (PTEs). Each PTE points to a 4 KB physical page.



2.3.3 Entry format

A PDE or PTE is a 32-bit value. The upper 20 bits are the physical page index of the page table (for a PDE) or of the page (for a PTE). The lower 12 bits are permission flags:

Flag	Value	Meaning
`PTE_P`	`0x001`	present: the entry is valid
`PTE_W`	`0x002`	writable
`PTE_U`	`0x004`	accessible from user mode
`PTE_G`	`0x100`	global (kept in the TLB across CR3 changes)

$$\text{entry} = (\text{page index} \ll 12) \mid \text{flags}, \qquad \text{address in an entry} = \text{entry} \;\&\; \texttt{0xFFFFF000}.$$

For example, the PDE for physical page 100 with `PTE_P | PTE_W | PTE_U` is $100 \times 4096 + 7 = 409607$.

2.3.4 Page structures in mCertiKOS

```
unsigned int *PDirPool[NUM_IDS][1024];   // one page directory per process
unsigned int  IDPTbl[1024][1024];        // identity page tables, shared by all
```

- `PDirPool[p]` is the page directory of process `p`. Its entries are declared as pointers, but they hold PDE values (address plus flags); cast between `unsigned int` and `unsigned int *` as needed.
- `IDPTbl` contains 1024 page tables that together map every virtual address to the same physical address (an *identity map*). The kernel's part of every page directory points into `IDPTbl`.
- The page tables of the user part are normal physical pages, allocated on demand (experiment 3).

> **Note.** Paging is still off in this experiment. The page structures are only filled in; the processor uses them from experiment 3 onwards.

2.4 Files and interfaces

Layer	Functions it may call (**import.h**)
MContainer	`pmem_init`, `get_nps`, `at_is_norm`, `at_is_allocated`, `palloc`, `pfree`
MPTIntro	`set_cr3(unsigned int **pdir)`

2.5 Tasks

Task 2.1: Memory containers (MContainer)

File: kern/pmm/MContainer/MContainer.c.

```
void container_init(unsigned int mbi_addr);
```

After the call to `pmem_init`, compute `real_quota`: the number of pages that are normal and not allocated (loop over all pages from 0 to `get_nps() - 1`). The rest of the function is provided.

```
unsigned int container_get_parent(unsigned int id);
```

```
unsigned int container_get_nchildren(unsigned int id);
```

```
unsigned int container_get_quota(unsigned int id);
```

```
unsigned int container_get_usage(unsigned int id);
```

Return the corresponding field of `CONTAINER[id]`.

`unsigned int container_can_consume(unsigned int id, unsigned int n);`
Return 1 if process `id` can use `n` more pages without exceeding its quota (`usage + n <= quota`), and 0 otherwise.

`unsigned int container_split(unsigned int id, unsigned int quota);`
The index of the child and the check against `NUM_IDS` are provided. Initialize the child's container (`used = 1`, the given `quota`, `usage = 0`, `parent = id`, `nchildren = 0`), add `quota` to the parent's usage, and increment the parent's number of children. Return the child's index (provided).

`unsigned int container_alloc(unsigned int id);`
If one more page fits in the quota of process `id`, increment its usage, allocate a page with `palloc`, and store the page index in `page_index`. Otherwise leave `page_index` as 0 (failure).

`void container_free(unsigned int id, unsigned int page_index);`
If the page is allocated, free it with `pfree` and decrement the usage of process `id` (if it is greater than 0).

> **Caution.** Keep the provided `spinlock_acquire` and `spinlock_release` calls and write your code between them. Do not return between the two calls.

Check: `make LAB=2 TEST=1`; the *MContainer* group must pass.

Task 2.2: Page directory and page table entries (MPTIntro)

File: kern/vmm/MPTIntro/MPTIntro.c. In the following, `proc` is `proc_index`, `pde` is `pde_index`, and `pte` is `pte_index`.

`void set_pdir_base(unsigned int index);`
Load the page directory of process `index` into CR3: `set_cr3(PDirPool[index])`.

`unsigned int get_pdir_entry(unsigned int proc, unsigned int pde);`
Return `PDirPool[proc][pde]` as an `unsigned int`.

`void set_pdir_entry(unsigned int proc, unsigned int pde, unsigned int page_index);`
Set the PDE to the address of physical page `page_index` with the flags `PTE_P | PTE_W | PTE_U` (the constant `PT_PERM_PTU`).

`void set_pdir_entry_identity(unsigned int proc, unsigned int pde);`
Set the PDE to the address of the identity page table `IDPTbl[pde]`, with the flags `PT_PERM_PTU`.

`void rmv_pdir_entry(unsigned int proc, unsigned int pde);`
Set the PDE to 0.

`unsigned int get_ptbl_entry(unsigned int proc, unsigned int pde, unsigned int pte);`
The page table address is the PDE with its lower 12 bits cleared. Return entry `pte` of that page table.

`void set_ptbl_entry(unsigned int proc, unsigned int pde, unsigned int pte, unsigned int page_index, unsigned int perm);`
Set entry `pte` of that page table to `(page_index << 12) | perm`.

`void set_ptbl_entry_identity(unsigned int pde, unsigned int pte, unsigned int perm);`

Set `IDPTbl[pde][pte]` so that it maps the virtual address `(pde « 22) | (pte « 12)` to the same physical address, with the flags `perm`.

```
void rmv_ptbl_entry(unsigned int proc, unsigned int pde, unsigned int pte);
```

Set entry `pte` of that page table to 0.

Check: the *MPTIntro* group must pass.

2.6 Testing

```
make LAB=2 TEST=1
```

```
Testing the MContainer layer...
test 1 passed.
test 2 passed.
All tests passed.

Testing the MPTIntro layer...
test 1 passed.
test 2 passed.
All tests passed.

================================================
LAB 2 RESULT: 2 / 2 test groups passed.
================================================
```

2.7 Evaluation

Item	Requirement	Marks
MContainer	test group passes	4
MPTIntro	test group passes	4
Viva	correct answers to the viva questions	2
	Total	**10**

2.8 Viva questions

1. Explain the value 409607 used in the MPTIntro tests.
2. Why are `PTE_P`, `PTE_W` and `PTE_U` all set in a page directory entry?
3. Why does `container_split` increase the *parent's* usage?
4. How much memory does one page table map? How much does a whole page directory map?
5. Why can the kernel part of the page directories of all processes share the same page tables (`IDPTbl`)?

EXPERIMENT 3

Virtual Memory Management

Experiment at a glance

Goal: Build complete page structures for all processes, map and unmap pages on demand, and turn on paging.
Prerequisites: Experiments 1 and 2 complete.
Files: kern/vmm/MPTOp/MPTOp.c, kern/vmm/MPTComm/MPTComm.c, kern/vmm/MPTKern/MPTKern.c, kern/vmm/MPTNew/MPTNew.c
Run the tests: `make LAB=3 TEST=1`
Demonstration: `make LAB=3 && make qemu-nox` reaches the monitor with paging on
Schedule: one lab session of 100 minutes (lab week 5); evaluated at the end of the session
Marks: 10 (tests 7, demonstration 1, viva 2)

3.1 Objectives

After this experiment you will be able to:

- work with page table entries through virtual addresses;
- set up an identity map for the kernel in every address space;
- allocate page tables on demand and map pages for a process;
- explain what happens when paging is turned on.

3.2 Pre-lab work

1. Which page directory indices correspond to the kernel part of the address space (below `VM_USERLO` and from `VM_USERHI`)?
2. Write a C expression that computes the page directory index of a virtual address `va`, and one for its page table index.
3. When the kernel turns paging on, it keeps running at the same addresses. Why does this work only if the kernel is identity-mapped?

3.3 Background

3.3.1 Kernel and user parts of an address space

Every process has its own page directory (`PDirPool[p]`). In all of them, the *kernel part* must be the same identity map, so that the kernel code and data are always at their physical addresses, whichever process is running. With 4 MB per page directory entry (1024 pages of 4 KB):

Page directory indices	Addresses	Mapped to
0 – 255	[0, VM_USERLO)	identity page tables (kernel)
256 – 959	[VM_USERLO, VM_USERHI)	user pages, allocated on demand
960 – 1023	[VM_USERHI, 4 GB)	identity page tables (kernel, devices)

Define `VM_USERLO_PDE` = `VM_USERLO / (PAGESIZE * 1024)` = 256 and `VM_USERHI_PDE` = 960. These constants are defined at the top of the files of this experiment.

The kernel itself (process 0) uses the identity map for the *whole* address space, so that it can access all physical memory directly.

3.3.2 Identity page tables

The identity page tables `IDPTbl` map every address to itself. Their permissions are:

- `PTE_P | PTE_W | PTE_G` for the kernel part. The *global* bit keeps these translations in the TLB when CR3 changes.
- `PTE_P | PTE_W` for the user part, without `PTE_U`, so that user programs cannot access them.

3.3.3 Allocating page tables on demand

A user page directory starts with all user entries empty. When the kernel maps a page at virtual address *va* for a process and the page directory entry of *va* is empty, it first allocates a physical page for a new page table (charged to the process's container), clears its 1024 entries, and stores it in the page directory. Then it sets the page table entry.

3.3.4 Error value

Functions that return a page index use the constant `MagicNumber` (lib/x86.h, value 1048577, larger than any page index) to report an error.

3.3.5 Turning paging on

After `pdir_init_kern`, the provided function `paging_init` (MPTInit) loads the page directory of process 0 into CR3 and enables paging. Because process 0 is identity-mapped, the next instruction is fetched from the same address and the kernel keeps running.

3.4 Files and interfaces

Layer	Functions it may call (**import.h**)
MPTOp	`container_init`, all the MPTIntro functions of experiment 2
MPTComm	`idptbl_init`, `container_alloc`, `container_free`, `set_pdir_entry_identity`, `rmv_pdir_entry`, `rmv_ptbl_entry`, `get_pdir_entry_by_va`, `set_pdir_entry_by_va`, `rmv_pdir_entry_by_va`
MPTKern	`pdir_init`, `set_pdir_entry_identity`, `alloc_ptbl`, `get_pdir_entry_by_va`, `get_ptbl_entry_by_va`, `set_ptbl_entry_by_va`, `rmv_ptbl_entry_by_va`
MPTNew	`container_alloc`, `container_split`, `map_page`

3.5 Tasks

Task 3.1: Operations by virtual address (MPTOp)

File: kern/vmm/MPTOp/MPTOp.c. Compute the page directory index and the page table index of `vaddr` as in experiment 2, and call the MPTIntro functions.

`unsigned int get_ptbl_entry_by_va(unsigned int proc, unsigned int vaddr);`
If the page directory entry of `vaddr` is not 0, return the page table entry of `vaddr`; otherwise return 0.

`unsigned int get_pdir_entry_by_va(unsigned int proc, unsigned int vaddr);`
Return the page directory entry of `vaddr`.

`void rmv_ptbl_entry_by_va(unsigned int proc, unsigned int vaddr);`
If the page directory entry of `vaddr` is not 0, remove the page table entry of `vaddr`.

`void rmv_pdir_entry_by_va(unsigned int proc, unsigned int vaddr);`
Remove the page directory entry of `vaddr`.

`void set_ptbl_entry_by_va(unsigned int proc, unsigned int vaddr, unsigned int page_index, unsigned int perm);`
Set the page table entry of `vaddr` to physical page `page_index` with `perm`. The page table already exists.

`void set_pdir_entry_by_va(unsigned int proc, unsigned int vaddr, unsigned int page_index);`
Set the page directory entry of `vaddr` to the page table in physical page `page_index`.

`void idptbl_init(unsigned int mbi_addr);`
After `container_init`, fill all 1024×1024 identity page table entries with `set_ptbl_entry_identity`: permission `PTE_P | PTE_W | PTE_G` if the page directory index is in the kernel part, `PTE_P | PTE_W` otherwise.

Task 3.2: Common page structure operations (MPTComm)

File: kern/vmm/MPTComm/MPTComm.c.

`void pdir_init(unsigned int mbi_addr);`
After `idptbl_init`, for every process (0 to `NUM_IDS - 1`) and every page directory index: map the entry to the identity page table if it is in the kernel part, and remove it otherwise.

`unsigned int alloc_ptbl(unsigned int proc, unsigned int vaddr);`
Allocate a page for process `proc` with `container_alloc`. If that fails, return 0. Otherwise, register the page as the page table of `vaddr` with `set_pdir_entry_by_va`, clear all 1024 entries of the new page table with `rmv_ptbl_entry`, and return the page index.

`void free_ptbl(unsigned int proc, unsigned int vaddr);`
The page index of the page table is the page directory entry of `vaddr` shifted right by 12. Remove the page directory entry, then free the page with `container_free`.

Task 3.3: Kernel page structure and mapping (MPTKern)

File: kern/vmm/MPTKern/MPTKern.c.

`void pdir_init_kern(unsigned int mbi_addr);`
After `pdir_init`, also map the *user* part (indices `VM_USERLO_PDE` to `VM_USERHI_PDE -`

1) of process 0 to the identity page tables, so that process 0 has an identity map of the whole address space.

```
unsigned int map_page(unsigned int proc, unsigned int vaddr, unsigned int
page_index, unsigned int perm);
```

If the page directory entry of `vaddr` is 0, allocate a page table with `alloc_ptbl`. If that fails, return `MagicNumber`. Then set the page table entry of `vaddr` to `page_index` with `perm`, and return the page index of the page table (the page directory entry shifted right by 12, or the value returned by `alloc_ptbl`).

```
unsigned int unmap_page(unsigned int proc, unsigned int vaddr);
```

Read the page table entry of `vaddr`. If it is not 0, remove it. Return the entry that was read. Do not free the page table.

Task 3.4: Allocating a page on demand (MPTNew)

File: kern/vmm/MPTNew/MPTNew.c.

```
unsigned int alloc_page(unsigned int proc, unsigned int vaddr, unsigned int
perm);
```

Allocate a physical page for process `proc` with `container_alloc`. If that fails, return `MagicNumber`. Otherwise map it at `vaddr` with `perm` using `map_page`, and return the result of `map_page`.

In experiment 5, the page fault handler calls `alloc_page` when a user program touches an address that is not mapped yet.

3.6 Testing

```
make LAB=3 TEST=1
```

```
Testing the MPTOp layer...
test 1 passed.
All tests passed.

Testing the MPTComm layer...
test 1 passed.
test 2 passed.
All tests passed.

Testing the MPTKern layer...
test 1 passed.
test 2 passed.
All tests passed.

Testing the MPTNew layer...
test 1 passed.
All tests passed.

================================================
LAB 3 RESULT: 4 / 4 test groups passed.
================================================
```

> **Note.** In test mode the kernel calls `pdir_init_kern` but does not turn paging on, so that the tests can inspect all page structures.

3.7 Demonstration: running with paging on

```
make LAB=3
make qemu-nox
```

Now the kernel calls `paging_init`, which turns paging on. If your page structures are correct, the kernel reaches the monitor:

```
[D] kern/init/init.c:...: Kernel initialized.
[D] kern/init/init.c:...: In kernel main.

***************************************
Welcome to the mCertiKOS kernel monitor!
***************************************
Type 'help' for a list of commands.
$>
```

If QEMU restarts, freezes, or reports a fault instead, the kernel's identity map is wrong.

3.8 Evaluation

Item	Requirement	Marks
MPTOp	test group passes	2
MPTComm	test group passes	2
MPTKern	test group passes	2
MPTNew	test group passes	1
Paging on	the demonstration reaches the monitor	1
Viva	correct answers to the viva questions	2
	Total	**10**

3.9 Viva questions

1. Why must the kernel part of every page directory be an identity map?
2. What does `map_page` do when the page table for `vaddr` does not exist yet?
3. Why does `unmap_page` not free the page table?
4. Why do the identity page tables for the user part not have the `PTE_U` bit?
5. What is the purpose of the `PTE_G` bit?

EXPERIMENT 4

Kernel Threads and Scheduling

Experiment at a glance

Goal: Create kernel threads, manage their control blocks and queues, and switch between them with a round-robin scheduler.
Prerequisites: Experiments 1–3 complete; doubly linked lists.
Files: kern/thread/PKCtxNew/PKCtxNew.c, kern/thread/PTCBInit/PTCBInit.c, kern/thread/PTQueueInit/PTQueueInit.c, kern/thread/PThread/PThread.c
Run the tests: `make LAB=4 TEST=1`
Schedule: one lab session of 100 minutes (lab week 6); evaluated at the end of the session
Marks: 10 (tests 8, viva 2)

4.1 Objectives

After this experiment you will be able to:

- explain what a thread's kernel context is and how a context switch works;
- implement an index-based doubly linked queue;
- implement thread creation and round-robin yielding.

4.2 Pre-lab work

1. Read kern/thread/PKCtxIntro/cswitch.S (provided). Which registers does it save, and in which order are they stored in `struct kctx`?
2. Draw the queue $2 \leftrightarrow 3 \leftrightarrow 4$ (head 2, tail 4) with the `prev`/`next` values of each thread. Then draw it again after removing thread 3.
3. Why does a new thread's stack pointer point to the *highest* address of its stack?

4.3 Background

4.3.1 Threads and kernel contexts

A *thread* is an independent flow of execution with its own stack. mCertiKOS supports `NUM_IDS` = 64 threads, numbered 0 to 63. The thread number is also the process id and the container index. Thread 0 is the kernel's initial thread.

When a thread stops running, the kernel saves its *kernel context*: the registers the C compiler expects to be preserved across a function call, plus the stack pointer and the address where execution resumes.

```
struct kctx {          // kern/thread/PKCtxIntro/PKCtxIntro.c
```

```
    void *esp;          // stack pointer
    unsigned int edi;
    unsigned int esi;
    unsigned int ebx;
    unsigned int ebp;
    void *eip;          // where the thread continues
};
struct kctx kctx_pool[NUM_IDS];
```

`kctx_switch(from, to)` calls the provided assembly function `cswitch`. It saves the current registers into `kctx_pool[from]`, loads the registers of `kctx_pool[to]`, and returns, so execution continues in thread `to`. A *new* thread has never run, so its context only needs `esp` (the top of its kernel stack) and `eip` (its entry function).

Every thread has a kernel stack `proc_kstack[pid]` (lib/kstack.h). The top of the stack is the address `proc_kstack[pid].kstack_hi`. On x86 the stack grows towards lower addresses, so a new thread starts with `esp` at the top.

4.3.2 Thread control blocks (TCBs)

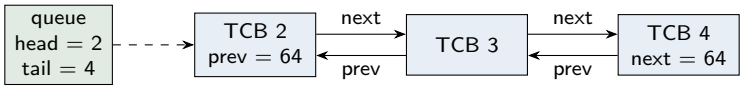
```
struct TCB {                    // kern/thread/PTCBIntro/PTCBIntro.c (simplified)
    t_state state;              // TSTATE_READY, TSTATE_RUN, TSTATE_SLEEP, TSTATE_DEAD
    unsigned int cpuid;         // the CPU the thread runs on
    unsigned int prev;          // previous thread in its queue
    unsigned int next;          // next thread in its queue
    ...
};
```

`tcb_init_at_id(pid)` sets a TCB to its initial values: state `TSTATE_DEAD` and `prev = next = NUM_IDS`. The value `NUM_IDS` means "no thread" (like `NULL`).

4.3.3 Thread queues

Threads are linked into queues through the `prev` and `next` fields of their TCBs. A queue only stores the ids of its first and last threads:

```
struct TQueue { unsigned int head; unsigned int tail; };
struct TQueue TQueuePool[NUM_IDS + NUM_CPUS];
```



There are `NUM_IDS + NUM_CPUS` queues. Queue `NUM_IDS + i` is the *ready queue* of CPU i. Until experiment 6 only CPU 0 is used, so the ready queue is `NUM_IDS + 0`. In your code, always write `NUM_IDS + get_pcpu_idx()`: it works for one CPU and for many.

4.3.4 Round-robin scheduling

- `thread_spawn` creates a thread and appends it to the ready queue.
- `thread_yield` moves the running thread to the end of the ready queue, takes the first thread of the queue, and switches to it.

The current thread id of each CPU is kept by `get_curid` and `set_curid`.

> **Note.** The scheduler state is protected by the spinlock `sched_lk`, because from experiment 6 on several CPUs schedule at the same time. The skeleton already acquires the lock; you must release it at the right place.

4.4 Files and interfaces

Layer	Functions it may call (import.h)
PKCtxNew	`container_can_consume`, `alloc_mem_quota`, `kctx_set_esp`, `kctx_set_eip`
PTCBInit	`paging_init`, `tcb_init_at_id`
PTQueueInit	`tcb_init`, `tcb_get_prev`, `tcb_set_prev`, `tcb_get_next`, `tcb_set_next`, `tqueue_get_head`, `tqueue_set_head`, `tqueue_get_tail`, `tqueue_set_tail`, `tqueue_init_at_id`
PThread	`kctx_new`, `kctx_switch`, `tcb_set_state`, `tcb_set_cpu`, `tqueue_init`, `tqueue_enqueue`, `tqueue_dequeue`, `get_curid`, `set_curid`, `get_pcpu_idx`

4.5 Tasks

Task 4.1: Creating a kernel context (PKCtxNew)

File: kern/thread/PKCtxNew/PKCtxNew.c.

```
unsigned int kctx_new(void *entry, unsigned int id, unsigned int quota);
```

If process `id` can give `quota` pages (`container_can_consume`), create the child with `alloc_mem_quota(id, quota)`, which returns the child's id. If the id is not `NUM_IDS`, set the child's `esp` to `proc_kstack[pid].kstack_hi` and its `eip` to `entry`. Return the child's id, or `NUM_IDS` on any error.

Task 4.2: Initializing the TCBs (PTCBInit)

File: kern/thread/PTCBInit/PTCBInit.c.

```
void tcb_init(unsigned int mbi_addr);
```

After `paging_init`, initialize the TCB of every thread (0 to `NUM_IDS - 1`) with `tcb_init_at_id`.

Task 4.3: Thread queues (PTQueueInit)

File: kern/thread/PTQueueInit/PTQueueInit.c.

```
void tqueue_init(unsigned int mbi_addr);
```

After `tcb_init`, initialize all `NUM_IDS + NUM_CPUS` queues with `tqueue_init_at_id` (an empty queue has head and tail `NUM_IDS`).

```
void tqueue_enqueue(unsigned int chid, unsigned int pid);
```

Append thread `pid` at the tail of queue `chid`. If the queue is empty, `pid` becomes both head and tail and has no neighbours. Otherwise link it after the current tail and make it the new tail. Its `next` must be `NUM_IDS`.

```
unsigned int tqueue_dequeue(unsigned int chid);
```

Remove the head of queue `chid` and return it, or return `NUM_IDS` if the queue is empty. The next thread (if any) becomes the head, and its `prev` becomes `NUM_IDS`. If the queue becomes empty, set both head and tail to `NUM_IDS`. Reset `prev` and `next` of the removed thread to `NUM_IDS`.

```
void tqueue_remove(unsigned int chid, unsigned int pid);
```

Remove thread `pid` from anywhere in queue `chid`. Handle the four cases: `pid` is the only element, the head, the tail, or in the middle. Reset its `prev` and `next` to `NUM_IDS`.

> **Tip.** Draw the queue before and after each operation and write down which fields change. Most bugs are a forgotten update of the head or the tail.

Task 4.4: Spawning and yielding (PThread)

File: kern/thread/PThread/PThread.c. Let `rdq = NUM_IDS + get_pcpu_idx()` be the ready queue of the current CPU.

```
unsigned int thread_spawn(void *entry, unsigned int id, unsigned int quota);
```

Create the thread with `kctx_new`. If it succeeds, record the current CPU in its TCB (`tcb_set_cpu(pid, get_pcpu_idx())`), set its state to `TSTATE_READY`, and append it to `rdq`. Store the id in `pid` (returned by the skeleton), or leave it as `NUM_IDS` on failure. Keep your code between the lock calls.

```
void thread_yield(void);
```

The skeleton has already acquired `sched_lk`.

1. Set the current thread's state to `TSTATE_READY` and append it to `rdq`.
2. Dequeue the next thread from `rdq`, set its state to `TSTATE_RUN`, and make it the current thread (`set_curid`).
3. Release `sched_lk`.
4. If the new thread is different from the old one, call `kctx_switch(old, new)`.

Release the lock on *both* paths: whether or not you switch.

4.6 Testing

```
make LAB=4 TEST=1
```

```
Testing the PKCtxNew layer...
test 1 passed.
All tests passed.

Testing the PTCBInit layer...
test 1 passed.
All tests passed.

Testing the PTQueueInit layer...
test 1 passed.
test 2 passed.
All tests passed.

Testing the PThread layer...
```

```
test 1 passed.
All tests passed.

================================================
LAB 4 RESULT: 4 / 4 test groups passed.
================================================
```

The PTQueueInit test enqueues threads 2, 3 and 4, removes 3, dequeues 2, and checks every `prev`, `next`, head and tail value.

4.7 Evaluation

Item	Requirement	Marks
PKCtxNew	test group passes	2
PTCBInit	test group passes	1
PTQueueInit	test group passes	3
PThread	test group passes	2
Viva	correct answers to the viva questions	2
	Total	**10**

4.8 Viva questions

1. Using cswitch.S, explain how execution continues in the new thread after `kctx_switch`.
2. Why does `cswitch` save only `esp`, `edi`, `esi`, `ebx`, `ebp` and `eip`, and not `eax`, `ecx` or `edx`?
3. What happens in `thread_yield` when the current thread is the only ready thread?
4. Why must `sched_lk` be released before `kctx_switch` and not after?
5. Why are the queues built from thread ids instead of pointers?

EXPERIMENT 5

Processes, Traps and System Calls

Experiment at a glance

Goal: Run user processes in their own address spaces: pass system call arguments and results, implement the `spawn` and `yield` system calls, and route exceptions and interrupts.
Prerequisites: Experiments 1–4 complete.
Files: kern/trap/TSyscallArg/TSyscallArg.c, kern/trap/TSyscall/TSyscall.c, kern/trap/TTrapHandler/TTrapHandler.c
Run the tests: `make LAB=5 TEST=1`
Demonstration: `make LAB=5 && make qemu-nox`, then type `startuser`
Schedule: two lab sessions of 100 minutes (lab weeks 7–8): Tasks 5.1–5.2 in the first, Tasks 5.3–5.4 in the second; evaluated at the end of the second session
Marks: 10 (tests 4, demonstration 4, viva 2), bonus 2

5.1 Objectives

After this experiment you will be able to:

- explain how a user program enters the kernel through a trap;
- implement the system call calling convention using the trap frame;
- create user processes and switch between them;
- handle page faults by allocating memory on demand.

5.2 Pre-lab work

1. List the registers that carry the system call number, the arguments, the error number and the return values (Section 5.3.3).
2. What is the difference between an exception, a hardware interrupt and a system call? Give the trap number range of each in mCertiKOS.
3. A user program reads the address `0xe0000000` for the first time. Which trap happens, and which of the functions you implement in this experiment and in experiment 3 handle it?

5.3 Background

5.3.1 User processes

A *process* is a thread that runs a user program in user mode (ring 3), with its own page directory. The provided function `proc_create(elf_addr, quota)` (layer PProc):

1. creates a thread with `thread_spawn`;

2. loads the program's ELF image into the new address space;
3. prepares the user registers so that the first switch to the thread enters user mode at the program's entry point.

It returns the new process id, or `NUM_IDS` on failure. The user programs are compiled into the kernel image; there is no file system yet:

elf_id	Program	Source
1	ping	user/pingpong/ping.c
2	pong	user/pingpong/pong.c
3	ding	user/pingpong/ding.c
4	fstest	user/fstest/fstest.c (experiment 7)
5	fork_test	user/fork_test/fork_test.c (optional task)

5.3.2 Traps

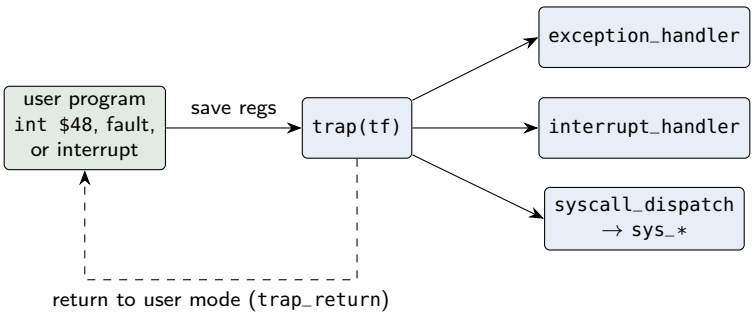
A *trap* transfers control from user mode to the kernel. There are three kinds:

Kind	Cause	Trap numbers
Exception	the program did something special: page fault (14), divide error (0), general protection fault (13)...	0–19, 30
Interrupt	a device needs attention: timer, keyboard, disk...	32–47 (`T_IRQ0 + IRQ_*`), 49–51
System call	the program executes `int $48`	48 (`T_SYSCALL`)

On a trap, the processor and the provided entry code save all user registers in a *trap frame* (`tf_t`, lib/trap.h) and call `trap(tf)` in TTrapHandler.c:

```
typedef struct pushregs {
    uint32_t edi, esi, ebp, oesp, ebx, edx, ecx, eax;
} pushregs;

typedef struct tf_t {
    pushregs regs;          // general registers of the user program
    uint16_t es, padding_es, ds, padding_ds;
    uint32_t trapno;        // which trap happened
    uint32_t err;           // error code (e.g. for page faults)
    uintptr_t eip;          // user instruction pointer
    ...                     // cs, eflags, esp, ss
} tf_t;
```



When the handler returns, the kernel restores the (possibly modified) registers from the trap frame and returns to the user program. A system call returns its results by *writing into the trap frame*.

5.3.3 System call convention

Register	On entry	On return
`eax`	system call number (argument 1)	error number (`E_SUCC` = 0 on success)
`ebx`	argument 2	return value 1
`ecx`	argument 3	return value 2
`edx`	argument 4	return value 3
`esi`	argument 5	return value 4
`edi`	argument 6	return value 5

For example, the user library function `spawn(elf_id, quota)` puts `SYS_spawn` in `eax`, `elf_id` in `ebx` and `quota` in `ecx`, and reads the new process id from `ebx`. The system call numbers and error numbers are listed in kern/lib/syscall.h.

5.3.4 Handlers

- **Exceptions.** A page fault (`T_PGFLT`) is handled by `pgflt_handler`. It reads the faulting address from register CR2 and calls `alloc_page` (experiment 3) to map a new page there; this is *demand paging*. Every other exception is fatal and goes to `default_exception_handler`, which prints the trap frame and stops the kernel.
- **Interrupts.** The timer interrupt calls `timer_intr_handler`, which acknowledges the interrupt and calls the scheduler (`sched_update`). After a time slice, the running process is preempted. A spurious interrupt (`T_IRQ0 + IRQ_SPURIOUS`) is ignored. Every other interrupt is acknowledged by `default_intr_handler`.

5.4 Files and interfaces

Layer	Functions it may call
TSyscallArg	only the trap frame `tf`
TSyscall	`syscall_get_arg1..6`, `syscall_set_errno`, `syscall_set_retval1..5`, `get_curid`, `container_can_consume`, `container_get_nchildren`, `proc_create`, `thread_yield`
TTrapHandler	`pgflt_handler`, `default_exception_handler`, `spurious_intr_handler`, `timer_intr_handler`, `default_intr_handler` (all in the same file)

5.5 Tasks

Task 5.1: System call arguments and results (TSyscallArg)

File: kern/trap/TSyscallArg/TSyscallArg.c. Implement the following functions according to the table in Section 5.3.3:

`unsigned int syscall_get_arg1(tf_t *tf); ... unsigned int syscall_get_arg6(tf_t *tf);`
Return `tf->regs.eax`, `tf->regs.ebx`, `tf->regs.ecx`, `tf->regs.edx`, `tf->regs.esi` and `tf->regs.edi` respectively.

`void syscall_set_errno(tf_t *tf, unsigned int errno);`
Store `errno` in `tf->regs.eax`.

`void syscall_set_retval1(tf_t *tf, unsigned int retval); ... syscall_set_retval5`
Store the value in `ebx`, `ecx`, `edx`, `esi` and `edi` respectively.

Task 5.2: The spawn and yield system calls (TSyscall)

File: kern/trap/TSyscall/TSyscall.c.

`void sys_spawn(tf_t *tf);`
Read `elf_id` (argument 2) and `quota` (argument 3), and let `curid = get_curid()`. Check the following, in this order. On an error, set the error number shown and return value 1 `NUM_IDS`, then return:

1. `curid` cannot consume `quota` pages: `E_EXCEEDS_QUOTA`;
2. `curid * MAX_CHILDREN + 1 + MAX_CHILDREN > NUM_IDS`: `E_MAX_NUM_CHILDEN_REACHED`;
3. `curid` already has `MAX_CHILDREN` children: `E_INVAL_CHILD_ID`;
4. `elf_id` is not 1–5: `E_INVAL_PID`. Otherwise, select the ELF address from the `_binary___obj_user_..._start` symbols declared above the function, following the table of elf ids;
5. `proc_create(elf_addr, quota)` returns `NUM_IDS`: `E_INVAL_PID`.

On success, set the error number to `E_SUCC` and return value 1 to the new process id.

`void sys_yield(tf_t *tf);`
Call `thread_yield()` and set the error number to `E_SUCC`.

Task 5.3: Routing exceptions and interrupts (TTrapHandler)

File: kern/trap/TTrapHandler/TTrapHandler.c.

`void exception_handler(tf_t *tf);`
If `tf->trapno` is `T_PGFLT`, call `pgflt_handler(tf);` otherwise call `default_exception_handler(tf)`.

`void interrupt_handler(tf_t *tf);`
Use a `switch` on `tf->trapno`: `T_IRQ0 + IRQ_SPURIOUS` calls `spurious_intr_handler()`, `T_IRQ0 + IRQ_TIMER` calls `timer_intr_handler()`, and anything else calls `default_intr_handler()`. (Experiment 7 adds a case for the disk.)

Task 5.4: Optional (bonus): fork with copy-on-write

`sys_fork` creates a copy of the calling process. Instead of copying all its memory, share the user pages between parent and child:

1. make the shared pages read-only in both processes and mark them with `PTE_COW`;
2. in `pgflt_handler`, the `PTE_COW` branch is already detected: when a process writes to such a page, copy it to a new page and map the copy writable;
3. return the child's id to the parent and 0 to the child.

Then add `spawn(5, 1000);` to user/idle/idle.c to run user/fork_test.

5.6 Testing

Part A: tests

```
make LAB=5 TEST=1
```

```
Testing the TSyscallArg layer...
test 1 passed.
test 2 passed.
All tests passed.

Testing the TSyscall (sys_spawn) layer...
test 1 passed.
test 2 passed.
All tests passed.

================================================
LAB 5 RESULT: 2 / 2 test groups passed.
================================================
```

The TSyscall test calls `sys_spawn` with an invalid `elf_id`, with a quota that is too large, and with valid arguments, and checks the error numbers and the new process.

Part B: demonstration

```
make LAB=5
make qemu-nox
```

At the monitor prompt, type `startuser`. This starts the `idle` process, which spawns `ping`, `pong` and `ding`. Each of them writes a different value to the same virtual address `0xe0000000`, yields, and reads the value back. Because every process has its own address space, each must read back its *own* value. The order of the lines can differ because of preemption:

```
$> startuser
[D] kern/lib/monitor.c:76: process idle 1 is created.
Start user-space ...
idle
ping in process 4.
pong in process 5.
ding in process 6.
ping started.
ping: the value at address e0000000: 0
ping: writing the value 100 to the address e0000000
pong started.
pong: the value at address e0000000: 0
ding started.
```

```
ding: the value at address e0000000: 0
ding: writing the value 300 to the address e0000000
ping: the new value at address e0000000: 100
pong: writing the value 200 to the address e0000000
ding: the new value at address e0000000: 300
pong: the new value at address e0000000: 200
```

5.7 Evaluation

Item	Requirement	Marks
TSyscallArg	test group passes	2
sys_spawn	test group passes	2
User processes	`startuser` output as above; each process reads back its own value; no panic	4
Viva	correct answers to the viva questions	2
	Total	**10**
Bonus	`fork_test` runs correctly with copy-on-write	*+2*

5.8 Viva questions

1. Why does the first access to `0xe0000000` cause a page fault, and which functions handle it?
2. Why do the three processes see different values at the same virtual address?
3. How does a system call return a value to the user program?
4. What would happen without the timer interrupt if a process never called `yield`?
5. Why does `sys_spawn` check the number of children before calling `proc_create`?

EXPERIMENT 6

Multiprocessor and Synchronization

Experiment at a glance

Goal: Run the kernel on several CPUs: register trap handlers per CPU, implement sleep and wakeup, and build a bounded buffer for producer and consumer processes.
Prerequisites: Experiments 1–5 complete; spinlocks and condition synchronization from the lectures.
Files: kern/trap/TTrapInit/TTrapInit.c, kern/thread/PThread/PThread.c, kern/trap/TSyscall/bbuf.c, kern/trap/TSyscall/TSyscall.c
Run the tests: `make LAB=6 TEST=1`
Demonstration: `make LAB=6 && make qemu-nox`
Schedule: two lab sessions of 100 minutes (lab weeks 9–10): Task 6.1 in the first, Tasks 6.2–6.4 in the second; evaluated at the end of the second session
Marks: 10 (tests 6, demonstration 2, viva 2)

6.1 Objectives

After this experiment you will be able to:

- explain how a multiprocessor kernel keeps per-CPU and shared state;
- implement sleep and wakeup without losing wakeups;
- implement a blocking bounded buffer;
- synchronize producer and consumer processes running on different CPUs.

6.2 Pre-lab work

1. What is a race condition? Give an example with two CPUs that both execute `count++`.
2. Explain the *lost wakeup* problem: what goes wrong if a thread checks a condition, then a wakeup happens, and only then the thread goes to sleep?
3. A bounded buffer has room for 10 items and contains 10. A producer wants to add one item. What must it do, and who will let it continue?

6.3 Background

6.3.1 The multiprocessor kernel

From this experiment on, the kernel starts all CPUs of the machine (QEMU emulates 4). CPU 0, the *bootstrap processor*, starts the others, the *application processors*. Each CPU has:

- its own current thread (`get_curid` returns the thread of the CPU that calls it);

- its own ready queue `NUM_IDS + cpu`;
- its own trap handler table `TRAP_HANDLER[cpu][trapno]`.

Memory is shared. Every shared data structure is protected by a *spinlock* (lib/spinlock.h):

```
spinlock_t lk;
spinlock_init(&lk);
spinlock_acquire(&lk);   // waits until no other CPU holds lk
...                      // critical section
spinlock_release(&lk);
```

This is why the code of the earlier experiments holds locks: `mem_lock` in `palloc`, the container locks, and `sched_lk` in the scheduler.

6.3.2 Trap handler registration

In experiments 1–5, `trap()` routed traps by their numbers. From now on, it only looks up the function registered in the table of the current CPU:

```
handler = TRAP_HANDLER[get_pcpu_idx()][tf->trapno];
```

The function `trap_init(cpu_idx)` runs on every CPU at boot. It must fill the table of that CPU by calling `trap_handler_register(cpu_idx, trapno, handler)` for each trap number.

6.3.3 Sleep and wakeup

A thread that must wait for a condition, such as "the buffer is not empty", should not spin: that wastes a CPU. Instead it *sleeps* on a *channel*, which can be any address that identifies the condition. A thread that makes the condition true *wakes up* all threads sleeping on that channel.

```
void thread_sleep(void *chan, spinlock_t *lk);  // caller holds lk
void thread_wakeup(void *chan);
```

The condition is protected by a lock `lk`, so a waiting thread does:

```
spinlock_acquire(&lk);
while (!condition)
    thread_sleep(&chan, &lk);   // releases lk while sleeping, re-acquires it after
... use the data ...
spinlock_release(&lk);
```

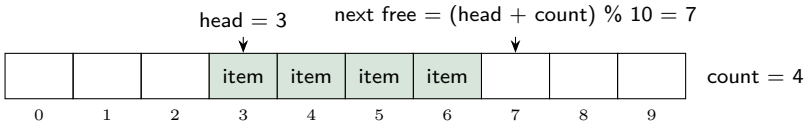
`thread_sleep` must release `lk` (otherwise no one could change the condition) and go to sleep *atomically*. Otherwise a wakeup could happen between the release and the sleep and be lost. mCertiKOS achieves this with `sched_lk`. `thread_sleep` acquires `sched_lk` *before* releasing `lk`. `thread_wakeup` also takes `sched_lk`, so it cannot run in the gap.

6.3.4 The bounded buffer

A bounded buffer is a FIFO queue with a fixed capacity of `BBUF_SIZE` items (10, defined in kern/trap/TSyscall/bbuf.h). It is stored as a circular array:

```
static struct {
    spinlock_t lock;
```

```
    unsigned int items[BBUF_SIZE];
    unsigned int head;   // index of the oldest item
    unsigned int count;  // number of items in the buffer
} bbuf;
```



- `bbuf_put(item)` waits while the buffer is full, stores the item at `(head + count) % BBUF_SIZE`, increments `count`, and wakes up the consumers.
- `bbuf_get()` waits while the buffer is empty, takes the item at `head`, advances `head`, decrements `count`, and wakes up the producers.

6.3.5 Producer and consumer processes

The user program `ping` calls the system call `produce()` 20 times, and `pong` calls `consume()` 10 times (user/pingpong/). The kernel creates two `ping` processes on CPU 1 and two `pong` processes on CPU 2. The system calls use the bounded buffer. Process `pid` produces the items `pid` $\times$ 1000 + 0, +1, +2, ..., so the output shows which process produced each consumed item.

6.4 Files and interfaces

Task	Functions you need
TTrapInit	`trap_handler_register`, `exception_handler`, `interrupt_handler`, `syscall_dispatch`; trap numbers in dev/intr.h
sleep/wakeup	`spinlock_acquire`, `spinlock_release`, `get_curid`, `set_curid`, `tcb_set_state`, `tcb_get_state`, `tcb_set_chan`, `tcb_get_chan`, `tcb_get_cpu`, `tqueue_enqueue`, `tqueue_dequeue`, `kctx_switch`, `get_pcpu_idx`
bounded buffer	`spinlock_acquire`, `spinlock_release`, `thread_sleep`, `thread_wakeup`
system calls	`bbuf_put`, `bbuf_get`, `get_curid`, `get_pcpu_idx`, `syscall_set_errno`, `KERN_INFO`

6.5 Tasks

Task 6.1: Registering trap handlers (TTrapInit)

File: kern/trap/TTrapInit/TTrapInit.c, function `trap_init(cpu_idx)`. For every `trapno` from 0 to 255, register the handler for CPU `cpu_idx`:

Trap numbers	Handler
`T_DIVIDE` ... `T_SIMD`, and `T_SECEV`	`exception_handler`
`T_IRQ0 + IRQ_TIMER` ... `T_IRQ0 + IRQ_RTC`, `T_IRQ0 + IRQ_MOUSE` ... `T_IRQ0 + IRQ_IDE2`, `T_IRQ0 + IRQ_ERROR`, `T_IRQ0 + IRQ_EHCI_2`, `T_LTIMER` ... `T_PERFCTR`	`interrupt_handler`
`T_SYSCALL`	`syscall_dispatch`

Leave all other trap numbers unregistered.

Task 6.2: Sleep and wakeup (PThread)

File: kern/thread/PThread/PThread.c.

```
void thread_sleep(void *chan, spinlock_t *lk);
```

1. Acquire `sched_lk`, then release `lk`.
2. Record `chan` in the current thread's TCB (`tcb_set_chan`) and set its state to `TSTATE_SLEEP`.
3. Dequeue the next thread from the ready queue of this CPU (`NUM_IDS + get_pcpu_idx()`), set it to `TSTATE_RUN`, and make it the current thread.
4. Release `sched_lk` and switch to the new thread with `kctx_switch`.
5. When the thread runs again (after a wakeup), clear its channel and re-acquire `lk`.

```
void thread_wakeup(void *chan);
```

With `sched_lk` held, find every thread that is in state `TSTATE_SLEEP` with channel `chan`. Set it to `TSTATE_READY` and append it to the ready queue of *its* CPU: `NUM_IDS + tcb_get_cpu(pid)`.

> **Caution.** Do not call `thread_yield` from `thread_sleep`: the sleeping thread must not be put back into the ready queue.

Task 6.3: The bounded buffer

File: kern/trap/TSyscall/bbuf.c. `bbuf_init` and `bbuf_count` are provided.

```
void bbuf_put(unsigned int item);
```

Acquire `bbuf.lock`. While the buffer is full, sleep on a "not full" channel. Store the item at `(head + count) % BBUF_SIZE` and increment `count`. Wake up the threads waiting on the "not empty" channel. Release the lock.

```
unsigned int bbuf_get(void);
```

Acquire `bbuf.lock`. While the buffer is empty, sleep on the "not empty" channel. Take the item at `head`, advance `head` (modulo `BBUF_SIZE`), and decrement `count`. Wake up the threads waiting on the "not full" channel. Release the lock and return the item.

> **Tip.** Use the addresses of two different fields of `bbuf` as the channels, for example `&bbuf.head` for "not full" and `&bbuf.count` for "not empty".

Task 6.4: Producer and consumer system calls (TSyscall)

File: kern/trap/TSyscall/TSyscall.c.

```
void sys_produce(tf_t *tf);
```

Let `pid = get_curid()`. The item is `pid * 1000 + nproduced[pid]`; increment `nproduced[pid]`. Put the item into the buffer, print `CPU <c>: Process <pid>: Produced <item>` with `KERN_INFO`, and set the error number to `E_SUCC`.

```
void sys_consume(tf_t *tf);
```

Take an item from the buffer, print `CPU <c>: Process <pid>: Consumed <item>`, and set the error number to `E_SUCC`. Here `<c>` is `get_pcpu_idx()`.

6.6 Testing

Part A: tests

```
make LAB=6 TEST=1
```

```
Testing the TTrapInit layer...
test 1 passed.
All tests passed.

Testing the PThread (sleep/wakeup) layer...
test 1 passed.
All tests passed.

Testing the bounded buffer layer...
test 1 passed.
test 2 passed.
test 3 passed.
All tests passed.

=================================================
LAB 6 RESULT: 3 / 3 test groups passed.
=================================================
```

The tests check that:

- the trap handlers are registered;
- a thread sleeping on a channel is woken up only by a wakeup on that channel and gets its lock back;
- the buffer is FIFO, a consumer blocks on an empty buffer, and a producer blocks on a full buffer.

Caution. If the test run stops in the middle, `thread_sleep` (or the waiting loop in the buffer) probably does not switch to another thread.

Part B: demonstration

```
make LAB=6
make qemu-nox
```

The kernel reports 4 CPUs, creates `ping` processes 1 and 2 on CPU 1 and `pong` processes 4 and 5 on CPU 2, and they exchange items:

```
[BSP KERN] Number of CPUs in this system: 4.
...
CPU1: process ping1 1 is created.
CPU1: process ping2 2 is created.
CPU2: process pong1 4 is created.
CPU2: process pong2 5 is created.
CPU 1: Process 1: Produced 1000
CPU 1: Process 1: Produced 1001
CPU 1: Process 1: Produced 1002
CPU 1: Process 2: Produced 2000
CPU 2: Process 4: Consumed 1000
CPU 1: Process 2: Produced 2001
CPU 2: Process 4: Consumed 1001
...
```

Check that every item is consumed *once*, in the order in which it was produced. The producers make 40 items and the consumers take 20. When the buffer is full, the producers sleep forever. This is expected; exit QEMU with `Ctrl-a` `x`.

> **Note.** On a machine with fewer than 3 CPUs, the kernel prints `Fewer than 3 CPUs: running ping and pong on CPU 0` and runs one producer and one consumer on CPU 0.

6.7 Evaluation

Item	Requirement	Marks
TTrapInit	test group passes	1
Sleep/wakeup	test group passes	2
Bounded buffer	test group passes	3
Producer/consumer	Part B: items consumed in FIFO order, none lost or duplicated, no panic	2
Viva	correct answers to the viva questions	2
	Total	**10**

6.8 Viva questions

1. Why must `thread_sleep` acquire `sched_lk` before releasing `lk`?
2. Why does `bbuf_get` check the condition in a `while` loop and not with an `if`?
3. Why does every CPU have its own ready queue and its own trap handler table?
4. Why does `thread_wakeup` put a thread into the ready queue of the thread's CPU and not of the current CPU?
5. What would happen if `bbuf_put` called `thread_sleep` without holding `bbuf.lock`?

EXPERIMENT 7

Disk and File System

Experiment at a glance

Goal: Complete the file system: handle disk interrupts, look up directories and path names, and implement the file system calls.
Prerequisites: Experiments 1–6 complete, in particular `thread_sleep` and `thread_wakeup`.
Files: kern/trap/TTrapHandler/TTrapHandler.c, kern/fs/dir.c, kern/fs/path.c, kern/fs/sysfile.c
Run the tests: `make LAB=7 TEST=1` (or `make LAB=7 && make qemu-nox`)
Schedule: two lab sessions of 100 minutes (lab weeks 11–12): Tasks 7.1–7.3 in the first, Task 7.4 in the second; evaluated at the end of the second session
Marks: 10 (file system test suite 8, viva 2)

7.1 Objectives

After this experiment you will be able to:

- explain how a process waits for a disk request using interrupts;
- describe the layers of a Unix-like file system;
- implement directory lookup and path name resolution;
- implement the `read`, `write`, `close` and `fstat` system calls on top of the file layer.

7.2 Pre-lab work

1. What is an inode? What is the difference between an inode and a directory entry?
2. For the path `"///a//bb/c"`, list the path elements that `skipelem` returns one after the other.
3. Why can a process not simply wait in a loop until the disk has finished a request?

7.3 Background

7.3.1 Disk requests and interrupts

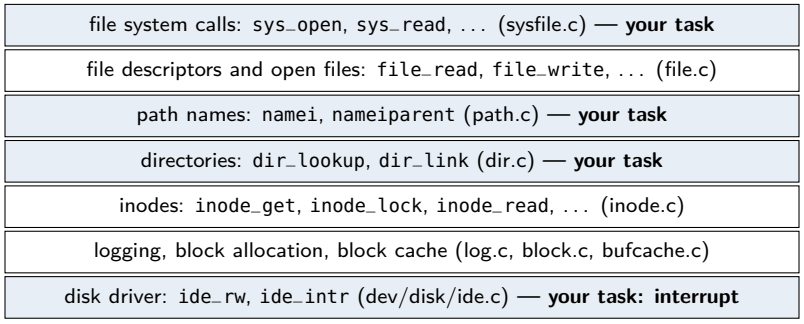
The second disk of QEMU, certikos_disk.img, holds the file system. To read or write a block, the file system calls `ide_rw(b)` (kern/dev/disk/ide.c). This function:

1. appends the request to the disk queue;
2. starts the disk if it is idle;
3. sleeps with `thread_sleep(b, &ide_lk)` until the request is complete.

When the disk finishes, it raises interrupt `T_IRQ0 + IRQ_IDE1`. The interrupt handler `ide_intr()` completes the request, wakes up the waiting process with `thread_wakeup(b)`, and starts the next request. While one process waits, the CPU runs other processes.

7.3.2 File system layers

The file system is a simplified version of the xv6 file system. It is organised in layers:



7.3.3 Inodes

An *inode* describes a file or a directory: its type (`T_FILE` or `T_DIR`), size, link count, and the disk blocks holding its data. In memory, inodes are cached in `struct inode` (kern/fs/inode.h).

`inode_get(dev, inum)`	return the in-memory inode `inum`, with a new reference (not locked, not read from disk yet)
`inode_dup(ip)`	take another reference to `ip`
`inode_lock(ip)`	lock `ip`, reading it from disk if necessary; required before using its fields or data
`inode_unlock(ip)`	unlock `ip`
`inode_put(ip)`	drop a reference
`inode_unlockput(ip)`	unlock, then drop a reference
`inode_read(ip, dst, off, n)`	read `n` bytes at offset `off`; returns the number of bytes read
`inode_write(ip, src, off, n)`	write `n` bytes at offset `off`; returns the number written

7.3.4 Directories

A directory is a file whose data is an array of directory entries:

```
#define DIRSIZ 14
struct dirent {
    uint16_t inum;          // 0 means the entry is free
    char name[DIRSIZ];      // not necessarily 0-terminated if 14 characters long
};
```

Compare names with `dir_namecmp(a, b)` (at most `DIRSIZ` characters). The size of the directory, `dp->size`, is a multiple of `sizeof(struct dirent)`.

7.3.5 Path names

`namei("/a/b/c")` starts at the root directory (inode `ROOTINO`) for absolute paths, or at the current directory otherwise. It looks up each element in turn: `a` in the root, `b` in `a`, and `c` in `b`.

`nameiparent(path, name)` stops one element early: it returns the directory containing the last element and copies the last element into `name`. Both use the helper `namex`.

7.3.6 Open files and file descriptors

Each process has a table of `NOFILE` = 16 open files in its TCB:

- `tcb_get_openfiles(pid)` returns the array of `struct file *`;
- `tcb_set_openfiles(pid, fd, f)` sets one entry.

A file descriptor `fd` is an index into this table; a free entry is 0. The file layer offers:

- `file_read(f, buf, n)` and `file_write(f, buf, n)`, which return the number of bytes transferred or −1;
- `file_stat(f, &st)`;
- `file_close(f)`.

7.3.7 Kernel and user buffers

A system call receives *user* virtual addresses. The kernel must not dereference them directly: they belong to the process's address space and may be invalid. Instead, it copies data between a kernel buffer and the user's memory:

```
size_t pt_copyin(uint32_t pid, uintptr_t uva, void *kva, size_t len);  // user ->
    kernel
size_t pt_copyout(void *kva, uint32_t pid, uintptr_t uva, size_t len); // kernel ->
     user
```

A valid user buffer lies completely within [`VM_USERLO`, `VM_USERHI`).

7.3.8 System call arguments

The user library (user/include/syscall.h) passes the arguments as follows. All four system calls return a value in return value 1, which is −1 on error.

System call	arg 2	arg 3	arg 4
`sys_read(fd, buf, n)`	`fd`	`buf` (user address)	`n`
`sys_write(fd, buf, n)`	`fd`	`buf` (user address)	`n`
`sys_close(fd)`	`fd`		
`sys_fstat(fd, st)`	`fd`	`st` (user address of `struct file_stat`)	

7.4 Tasks

Task 7.1: Disk interrupts (TTrapHandler)

File: kern/trap/TTrapHandler/TTrapHandler.c, function `interrupt_handler`. Add a case for `T_IRQ0 + IRQ_IDE1` that calls `ide_intr()` and then `intr_eoi()` to acknowledge the interrupt.
Check: with `make LAB=7 TEST=1`, `fstest` starts and prints its first lines. Without this

task, the kernel stops after `fstest in process 4.`

Task 7.2: Directories (dir.c)

File: kern/fs/dir.c. The caller holds the lock of `dp`.

```
struct inode *dir_lookup(struct inode *dp, char *name, uint32_t *poff);
```

For every offset `off` from 0 to `dp->size` in steps of `sizeof(struct dirent)`, read the entry with `inode_read` (panic if it does not return a whole entry). Skip free entries (`inum == 0`). If the name matches, store `off` in `*poff` (if `poff` is not 0) and return `inode_get(dp->dev, de.inum)`. Return 0 if no entry matches.

```
int dir_link(struct inode *dp, char *name, uint32_t inum);
```

If `name` is already in the directory (`dir_lookup`), drop the reference it returned (`inode_put`) and return −1. Otherwise find the first free entry, or the end of the directory, write a new entry (`strncpy` the name, set `inum`) with `inode_write`, and return 0.

Task 7.3: Path names (path.c)

File: kern/fs/path.c.

```
static char *skipelem(char *path, char *name);
```

Skip leading slashes. If the path is then empty, return 0. Otherwise copy the next element (up to the next `'/'` or the end) into `name`. If it has `DIRSIZ` or more characters, copy only `DIRSIZ - 1` of them. Terminate `name` with `'\0'`, skip the slashes after the element, and return the rest of the path. Examples: `skipelem("a/bb/c", name)` returns `"bb/c"` with `name = "a"`; `skipelem("///a//bb", name)` returns `"bb"`; `skipelem("a", name)` returns `""`; `skipelem("", name)` and `skipelem("////", name)` return 0.

```
static struct inode *namex(char *path, bool nameiparent, char *name);
```

The start inode is set up for you. Complete the body of the loop, which runs once per path element:

1. lock `ip`; if it is not a directory, unlock and put it (`inode_unlockput`) and return 0;
2. if `nameiparent` is true and this was the last element (`*path == '\0'`), unlock `ip` and return it;
3. look up `name` in `ip` with `dir_lookup`; if it is not found, `inode_unlockput(ip)` and return 0;
4. `inode_unlockput(ip)` and continue with the inode found.

Task 7.4: File system calls (sysfile.c)

File: kern/fs/sysfile.c. On every error, set return value 1 to −1 and the error number to `E_BADF`. On success, set the error number to `E_SUCC`.

```
static int fdalloc(struct file *f);
```

Find the first free entry `fd` in the open file table of the current process, store `f` there with `tcb_set_openfiles`, and return `fd`. Return −1 if the table is full.

```
void sys_read(tf_t *tf);
```

Check the file descriptor (between 0 and `NOFILE - 1`, and an open file) and that the user buffer is valid. Read with `file_read` into a kernel buffer and copy to the user with

`pt_copyout`. Transfer at most one page per step and repeat until `n` bytes are read or `file_read` returns fewer bytes than requested (end of file). Return the total number of bytes read.

```
void sys_write(tf_t *tf);
```

Check as for `sys_read`. Copy the user data into a kernel buffer with `pt_copyin` and write it with `file_write`, at most one page per step. Return the number of bytes written.

```
void sys_close(tf_t *tf);
```

Check the file descriptor, clear the entry in the open file table, call `file_close`, and return 0.

```
void sys_fstat(tf_t *tf);
```

Check the file descriptor and the user address. Fill a kernel `struct file_stat` with `file_stat` and copy it to the user with `pt_copyout`. Return 0.

> **Tip.** Declare the kernel buffer as a `static` array, for example `static char kbuf[NUM_IDS][PAGESIZE];`, and use `kbuf[pid]`. A 4 KB local array would overflow the kernel stack.

7.5 Testing

```
make LAB=7 TEST=1
```

The kernel starts `idle`, which spawns the test program `fstest`. Every test prints a line ending in `ok`. The last test, `bigdir`, takes some time. The output looks like this; after the small file test, only the result line of each test is shown:

```
idle
fstest in process 4.
*******usertests starting*******

=====test file usertests.ran does not exists=====
=====test file usertests.ran does not exists: ok
=====small file test=====
create small succeeded; ok, fd: 0
writes ok
open small succeeded ok
read succeeded ok
=====small file test ok=====
=====big files test=====
=====big files ok=====
=====many creates, followed by unlink; ok=====
=====rmdot ok=====
=====fourteen ok=====
=====bigfile test ok=====
=====subdir ok=====
=====linktest ok=====
=====unlinkread ok=====
=====dir vs file OK=====
=====empty file name OK=====
=====bigdir ok=====
*******end of tests*******
```

Note. `fstest` creates a file usertests.ran and refuses to run twice on the same disk. `make LAB=7 TEST=1`, `make qemu` and `make qemu-nox` always start from a fresh copy of the disk (newfs/certikos_disk_new.img), so simply run the command again.

7.6 Evaluation

Item	Requirement	Marks
Disk interrupt	`fstest` starts and prints its first lines	1
Basic files	`small file test ok` and `big files ok`	2
Directories	creates/unlink, `rmdot` and `fourteen` tests pass	2
Paths and links	`bigfile`, `subdir`, `linktest` and `unlinkread` pass	2
Complete suite	`dir vs file`, `empty file name` and `bigdir` pass; `end of tests` printed	1
Viva	correct answers to the viva questions	2
	Total	**10**

7.7 Viva questions

1. Trace what happens, from `sys_read` down to the disk, when a process reads a block that is not in the block cache.
2. In `namex`, why is a directory unlocked before the next one is locked?
3. Why does the kernel copy user buffers with `pt_copyin`/`pt_copyout` instead of using the user pointer?
4. What does `dir_link` do when the directory has no free entry?
5. Why must the disk interrupt handler call `intr_eoi()`?

APPENDIX A

x86 and mCertiKOS Reference

A.1 Important constants

Constant	Value	Meaning
`PAGESIZE`	4096	size of a page
`NUM_IDS`	64	number of threads/processes; also "no thread" in queues
`NUM_CPUS`	8	maximum number of CPUs
`MAX_CHILDREN`	3	maximum number of children per process
`VM_USERLO`	`0x40000000`	start of the user part of the address space
`VM_USERHI`	`0xF0000000`	end of the user part of the address space
`VM_USERLO_PI`, `VM_USERHI_PI`	`0x40000`, `0xF0000`	the same, as page indices
`VM_USERLO_PDE`, `VM_USERHI_PDE`	256, 960	the same, as page directory indices
`MagicNumber`	1048577	error value for functions returning a page index
`BBUF_SIZE`	10	capacity of the bounded buffer (exp. 6)
`NOFILE`	16	open files per process (exp. 7)
`DIRSIZ`	14	maximum length of a file name (exp. 7)

A.2 Address translation

Page index of an address	`addr » 12`
Address of a page	`page_index « 12`
Page directory index of a virtual address	`va » 22`
Page table index of a virtual address	`(va » 12) & 0x3FF`
Offset in the page	`va & 0xFFF`
Address stored in a PDE or PTE	`entry & 0xFFFFF000`
Page index stored in a PDE or PTE	`entry » 12`
Build an entry	`(page_index « 12) | flags`

A.3 Page table entry flags

Flag	Value	Meaning
PTE_P	0x001	present
PTE_W	0x002	writable
PTE_U	0x004	user-accessible
PTE_PWT	0x008	write-through
PTE_PCD	0x010	cache disabled
PTE_A	0x020	accessed (set by the processor)
PTE_D	0x040	dirty (set by the processor)
PTE_PS	0x080	4 MB page (not used)
PTE_G	0x100	global
PTE_COW	0x800	copy-on-write (software-defined, exp. 5 bonus)

A.4 Registers

eax, ebx, ecx, edx, esi, edi	general-purpose registers; carry system call arguments and results
esp, ebp	stack pointer and frame pointer
eip	instruction pointer
eflags	flags; bit FL_IF enables interrupts
CR0	control register; bit 31 enables paging
CR2	the address that caused the last page fault (rcr2())
CR3	physical address of the current page directory (set_cr3, rcr3())
CR4	control register; enables processor extensions

A.5 Trap numbers

No.	Name	Meaning	No.	Name	Meaning
0	T_DIVIDE	divide error	32	T_IRQ0+IRQ_TIMER	timer
1	T_DEBUG	debug	33	T_IRQ0+IRQ_KBD	keyboard
3	T_BRKPT	breakpoint	36	T_IRQ0+IRQ_SERIAL13	serial port
6	T_ILLOP	illegal opcode	39	T_IRQ0+IRQ_SPURIOUS	spurious
8	T_DBLFLT	double fault	46	T_IRQ0+IRQ_IDE1	disk
13	T_GPFLT	general protection	48	T_SYSCALL	system call
14	T_PGFLT	page fault	49	T_LTIMER	local APIC timer
19	T_SIMD	SIMD exception	50	T_LERROR	local APIC error
30	T_SECEV	security event	51	T_PERFCTR	performance counter

The full list is in kern/dev/intr.h.

Page fault error code

For a page fault, tf->err describes the cause:

- bit 0 (PFE_PR) is set for a protection violation, and clear for a missing page;
- bit 1 is set for a write access;

- bit 2 is set if the access came from user mode.

A.6 System calls

No.	Name	Arguments (arg 2, 3, ...) → result
0	SYS_puts	string address, length
1	SYS_spawn	elf_id, quota → process id
2	SYS_yield	–
3	SYS_open	path, flags → fd
4	SYS_close	fd
5	SYS_read	fd, buffer, n → bytes read
6	SYS_write	fd, buffer, n → bytes written
7	SYS_mkdir	path
8	SYS_chdir	path
9	SYS_link	old path, new path
10	SYS_unlink	path
11	SYS_stat	fd, stat buffer
12	SYS_fork	– → child id (0 in the child)
13	SYS_produce	–
14	SYS_consume	–

A.7 Error numbers

0	E_SUCC (no error)	23	E_EXCEEDS_QUOTA
1	E_MEM	24	E_MAX_NUM_CHILDEN_REACHED
3	E_INVAL_CALLNR	25	E_INVAL_CHILD_ID
4	E_INVAL_ADDR	26	E_NEXIST
5	E_INVAL_PID	27	E_CREATE
13	E_DISK_OP	29	E_BADF

The full list is in kern/lib/syscall.h. Always use the names, not the numbers.

A.8 Thread states

State	Value	Meaning
TSTATE_READY	0	in a ready queue, waiting for a CPU
TSTATE_RUN	1	running on a CPU
TSTATE_SLEEP	2	waiting on a channel (exp. 6)
TSTATE_DEAD	3	not used

APPENDIX B

Kernel Function Reference

This appendix lists the functions you implement or use, grouped by layer. **Bold** names are the functions you implement; the experiment number is given in brackets. A layer may only call the functions listed in its own import.h.

B.1 Physical memory (kern/pmm)

MATIntro

`get_nps() / set_nps(nps)`	number of physical pages
`at_is_norm(i)` [1]	1 if page `i` has permission > 1
`at_set_perm(i, perm)` [1]	set the permission, mark unallocated
`at_is_allocated(i)` [1]	1 if page `i` is allocated
`at_set_allocated(i, a)` [1]	set the allocation flag
`mem_lock() / mem_unlock()`	lock protecting the allocation table

MATInit and MATOp

`get_size(), get_mms(r), get_mml(r), is_usable(r)`	the physical memory map
`pmem_init(mbi_addr)` [1]	compute `nps`, initialize the allocation table
`palloc()` [1]	allocate a free normal page; 0 on failure
`pfree(i)` [1]	free page `i`

MContainer

`container_init(mbi_addr)` [2]	quota of process 0
`container_get_parent(id)`, **`..._nchildren(id)`**, **`..._quota(id)`**, **`..._usage(id)`** [2]	container fields
`container_can_consume(id, n)` [2]	1 if `usage + n <= quota`
`container_split(id, quota)` [2]	create a child container; returns its index
`container_alloc(id)` [2]	allocate a page within the quota; 0 on failure
`container_free(id, i)` [2]	free page `i` of process `id`

B.2 Virtual memory (kern/vmm)

MPTIntro

set_pdir_base(p) [2]	load the page directory of process `p` into CR3
get/set/rmv_pdir_entry(p, pde, ...) [2]	page directory entries
set_pdir_entry_identity(p, pde) [2]	point a PDE to `IDPTbl[pde]`
get/set/rmv_ptbl_entry(p, pde, pte, ...) [2]	page table entries
set_ptbl_entry_identity(pde, pte, perm) [2]	identity page table entries

MPTOp, MPTComm, MPTKern, MPTNew

get/set/rmv_ptbl_entry_by_va(p, va, ...) [3]	PTE of a virtual address
get/set/rmv_pdir_entry_by_va(p, va, ...) [3]	PDE of a virtual address
idptbl_init(mbi_addr) [3]	fill the identity page tables
pdir_init(mbi_addr) [3]	initial page directories of all processes
alloc_ptbl(p, va) [3]	allocate and install a page table; 0 on failure
free_ptbl(p, va) [3]	remove and free a page table
pdir_init_kern(mbi_addr) [3]	identity map of the whole space for process 0
map_page(p, va, i, perm) [3]	map page `i` at `va`; `MagicNumber` on failure
unmap_page(p, va) [3]	remove the mapping; returns the old PTE
alloc_page(p, va, perm) [3]	allocate and map a page; `MagicNumber` on failure
`alloc_mem_quota(id, quota)`	= `container_split`
`paging_init(mbi_addr)`	initialize everything and turn paging on (provided)

B.3 Threads (kern/thread)

`kctx_set_esp(pid, esp)`, `kctx_set_eip(pid, eip)`	set fields of a kernel context
`kctx_switch(from, to)`	switch to thread `to` (provided)
`kctx_new(entry, id, quota)` [4]	create a thread context; `NUM_IDS` on failure
`tcb_get/set_state`, `tcb_get/set_cpu`, `tcb_get/set_prev`, `tcb_get/set_next`	TCB fields
`tcb_get/set_chan(pid, ...)`	sleep channel of a thread
`tcb_init_at_id(pid)`	reset one TCB
`tcb_init(mbi_addr)` [4]	reset all TCBs
`tqueue_get/set_head`, `tqueue_get/set_tail`, `tqueue_init_at_id`	queue fields
`tqueue_init`, `tqueue_enqueue`, `tqueue_dequeue`, `tqueue_remove` [4]	thread queues
`get_curid()`, `set_curid(pid)`	current thread of this CPU
`get_pcpu_idx()`	index of this CPU
`thread_spawn(entry, id, quota)` [4]	create a ready thread
`thread_yield()` [4]	switch to the next ready thread
`thread_sleep(chan, lk)` [6]	sleep on `chan`, releasing `lk`
`thread_wakeup(chan)` [6]	wake up all threads sleeping on `chan`

B.4 Traps and system calls (kern/trap)

`syscall_get_arg1..6(tf)` [5]	system call arguments
`syscall_set_errno(tf, e)` [5]	error number
`syscall_set_retval1..5(tf, v)` [5]	return values
`sys_spawn(tf)`, `sys_yield(tf)` [5]	system calls
`exception_handler(tf)`, `interrupt_handler(tf)` [5, 7]	trap routing
`proc_create(elf_addr, quota)`	create a user process (provided)
`trap_init(cpu_idx)` [6]	register trap handlers of a CPU
`trap_handler_register(cpu, trapno, f)`	register one handler
`bbuf_put(item)`, `bbuf_get()` [6]	bounded buffer
`sys_produce(tf)`, `sys_consume(tf)` [6]	producer/consumer system calls

B.5 File system (kern/fs)

`dir_lookup(dp, name, poff)` [7]	find a directory entry
`dir_link(dp, name, inum)` [7]	add a directory entry
`skipelem(path, name), namex(...)` [7]	path name resolution
`fdalloc(f)` [7]	allocate a file descriptor
`sys_read, sys_write, sys_close, sys_fstat` [7]	file system calls
`inode_get, inode_dup, inode_lock, inode_unlock, inode_put, inode_unlockput`	inode references and locks
`inode_read, inode_write`	inode data
`file_read, file_write, file_stat, file_close`	open files
`tcb_get_openfiles(pid), tcb_set_openfiles(pid, fd, f)`	open file table
`pt_copyin(pid, uva, kva, n), pt_copyout(kva, pid, uva, n)`	copy between user and kernel memory
`ide_intr()`	disk interrupt handler (provided)

B.6 Library (kern/lib)

`dprintf(fmt, ...)`	print
`KERN_DEBUG, KERN_INFO, KERN_WARN, KERN_PANIC`	print with location / stop
`KERN_ASSERT(cond)`	stop the kernel if `cond` is false
`memset, memzero, memcpy, memmove, memcmp`	memory
`strcmp, strncmp, strncpy, strnlen, strchr`	strings
`ROUNDDOWN(a, n), ROUNDUP(a, n)`	rounding
`spinlock_init, spinlock_acquire, spinlock_release`	spinlocks
`rcr2(), rcr3(), read_ebp(), read_esp()`	read registers
`intr_eoi()`	acknowledge an interrupt

APPENDIX C

Debugging the Kernel

C.1 Print debugging

The quickest way to understand what your code does is to print values:

```
dprintf("palloc: index = %u, nps = %u\n", palloc_index, nps);
KERN_DEBUG("entry = 0x%08x\n", entry);   // also prints file and line
KERN_ASSERT(pid < NUM_IDS);              // stops the kernel if false
```

Remove the debugging output before your evaluation: it can make the expected output hard to read.

C.2 Reading a kernel panic

When the kernel stops with `KERN_PANIC` or a failed assertion, it prints the file and line:

```
[P] kern/lib/spinlock.c:47: Kernel assertion failed: kstack->magic == KSTACK_MAGIC
```

Look at that line, then find out who called the function, for example by adding `dprintf` calls or with GDB. A trap that is not handled prints the trap frame:

```
trapframe at 1a2f68
...
        trapno:     0000000e
        eip:        40000123
[P] ... Trap 14 @ 0x40000123.
```

Trap 14 is a page fault (Appendix A); `eip` is the instruction that caused it.

C.3 Debugging with GDB

GDB can stop the kernel at a function, run it step by step, and show variables.

1. In a first terminal, build and start QEMU in debugging mode. QEMU starts paused and waits for GDB:

   ```
   make LAB=1 TEST=1 QEMU=true     # build only (QEMU=true skips running it)
   make qemu-nox-gdb
   ```

2. In a second terminal, in the same mcertikos directory, start GDB. It reads .gdbinit, loads the kernel symbols and connects to QEMU:

   ```
   make gdb        # picks gdb, gdb-multiarch or (macOS) i386-elf-gdb
   ```

3. Set breakpoints and continue:

```
(gdb) break palloc
(gdb) continue
```

GDB command	Effect
`break f` / `b file.c:42`	stop at function `f` / at a line
`continue` (`c`)	run until the next breakpoint
`next` (`n`) / `step` (`s`)	next line / step into a function
`finish`	run until the current function returns
`print x` (`p x`), `p/x x`	print a variable (in hexadecimal)
`p AT[262144]`	print an array element or structure
`backtrace` (`bt`)	show the chain of function calls
`info registers`	show all registers
`x/8x addr`	show 8 words of memory at `addr`
`display x`	print `x` after every step
`delete`	delete all breakpoints

Note. The provided .gdbinit opens assembly and register windows. Type `layout src` to see the C source instead, and `tui disable` to return to the plain command line.

Tip. The build also writes the full disassembly of the kernel to obj/kern/kernel.asm and its symbol table to obj/kern/kernel.sym. When a panic shows an `eip`, search for that address in kernel.asm to find the function.

APPENDIX D

Troubleshooting

D.1 Build problems

`undefined reference to '__udivdi3' or '__umoddi3'`

The 32-bit compiler support library is missing. Install it with `sudo apt install gcc-multilib` on Ubuntu (Section I.3) or `sudo dnf install glibc-devel.i686 libgcc.i686` on Fedora (Section I.4). `bash setup.sh` checks this for you.

`boot block too large: 65536 bytes (max 446)`

Your copy of the project is older than the current archive. Recent toolchains (e.g. Fedora's) add a note section to every object file, and the old build copied it into the boot sector. Download the current archive, or in boot/boot0/Makefile.inc, boot/boot1/Makefile.inc and kern/init/Makefile.inc change `-O binary` to `-O binary -R .note.gnu.property -R .note.gnu.build-id -R .comment`. Then run `make clean`.

`'bool' cannot be defined via typedef`

The same problem may show as the error

```
two or more data types in declaration specifiers.
```

GCC 15 compiles C23 by default, where `bool` is a keyword. The current project passes `-std=gnu11`; in an older copy, add `-std=gnu11` at the start of the `CFLAGS` line in the Makefile and run `make clean`.

`missing separator` or other strange `make` errors on macOS

You are running the old GNU Make 3.81 that ships with macOS. Install GNU Make 4 with `brew install make` and put it first in your `PATH` (Section I.5); `make -version` must show 4.x.

`Permission denied` for `./make_image.py` or the scripts

The archive was unpacked by Windows or copied from a Windows folder, which drops the execute permissions. Unpack it again inside Ubuntu (WSL) into ~/os-lab, as in Section I.6.

`Error: Can't find an i386-elf version of GCC/binutils`

Your compiler cannot produce 32-bit code. On Linux, install `gcc-multilib`. On macOS, install `i686-elf-gcc` (Section I.5) and check that it is in your `PATH`.

`LAB must be one of 1 to 7, or 8 for the project`

You passed an invalid `LAB` value, for example `make LAB=9`.

`error: ... undeclared` in your code

You used a function or variable that is not available in this layer. Check the layer's `import.h` and the headers included at the top of the file. Do not add includes to reach other layers' data.

Compiler or linker warnings in files you did not change

The build silences the known harmless warnings where the installed tools allow it. With some compiler versions, warnings such as the following may still appear; they come from the provided code and are harmless:

- `array subscript 0 is outside array bounds`;
- `LOAD segment with RWX permissions`;
- `missing .note.GNU-stack section implies executable stack`.

Pay attention to the warnings in *your* files.

D.2 Run-time problems

QEMU prints nothing after `Start boot1`, or is very slow

QEMU runs without hardware acceleration. Check `/dev/kvm` and your membership of the `kvm` group (Section I.3). Without KVM the kernel still boots, but booting and the tests take longer; wait at least a minute before giving up.

`qemu-system-x86_64: command not found`

Install QEMU:

- Ubuntu and WSL: `sudo apt install qemu-system-x86`;
- Fedora: `sudo dnf install qemu-system-x86`;
- macOS: `brew install qemu`.

`Could not access KVM kernel module: Permission denied`

Add yourself to the `kvm` group and log in again: `sudo usermod -aG kvm $USER`.

How do I exit QEMU?

Press `Ctrl-a`, release, then press `x`. If you started QEMU with a window (`make qemu`), close the window.

QEMU restarts in a loop, or stops right after `Kernel initialized`

The kernel caused a fault it could not handle, often a wrong page table after paging is turned on (experiment 3) or a wrong context switch (experiment 4). Run the tests of the earlier experiments again, then use GDB (Appendix C).

The tests stop in the middle and nothing happens

A thread waits forever: typically a lock that is never released (a `return` between acquire and release), or `thread_sleep` not switching to another thread (experiment 6). Check every path through your function.

`already ran user tests (file usertests.ran exists)`

You ran `fstest` on a disk where it already ran. Start QEMU with `make qemu-nox` or `make LAB=7 TEST=1`; both copy a fresh disk image.

My changes have no effect

Check that you edited the file in the project you are building, and that you pass the right `LAB` value. If in doubt, run `make clean` and build again.

D.3 Getting help

When you ask your teacher or a classmate for help, show:

- the exact command you ran;
- the complete error message or output;
- what you have already tried.

APPENDIX E

Lab Record and Report Format

Unless your teacher says otherwise, keep a lab record with one short report per experiment. A report is not a copy of this book; it shows what *you* did and learned.

E.1 Report structure

1. **Title block:** experiment number and title, your name and student ID, date, and the names of your group members (if any).
2. **Objectives:** two or three sentences in your own words.
3. **Implementation:** for each task, the functions you wrote and a short explanation of how they work. Include the key lines of code, not whole files. For each design decision, explain the reason, for example why `palloc` stores the last index.
4. **Results:** the output of `make LAB=n TEST=1` (the result lines), and the output of the demonstration if there is one.
5. **Problems and solutions:** the bugs you met and how you found them.
6. **Answers:** your answers to the pre-lab questions.
7. **Conclusion:** what you learned, in a few sentences.

E.2 Checklist before the evaluation

- ☐ `make LAB=n TEST=1` shows the expected result line.
- ☐ The demonstration (if any) produces the expected output.
- ☐ The tests of the earlier experiments still pass.
- ☐ Debugging output has been removed.
- ☐ Your work is committed (`git status` shows no changes).
- ☐ You can explain every function you wrote.
- ☐ The README in the project lists your group members and what each of you implemented, if your teacher asks for it.

E.3 Experiment record sheet

Exp.	Title	Date	Tests	Viva	Signature
1	Physical Memory Allocator				
2	Memory Containers and Page Tables				
3	Virtual Memory Management				
4	Kernel Threads and Scheduling				
5	Processes, Traps and System Calls				
6	Multiprocessor and Synchronization				
7	Disk and File System				